# *Computer Organisation & Program Execution 2019*

1

## Digital Logic

Uwe R. Zimmer - The Australian National University

## *References for this chapter*

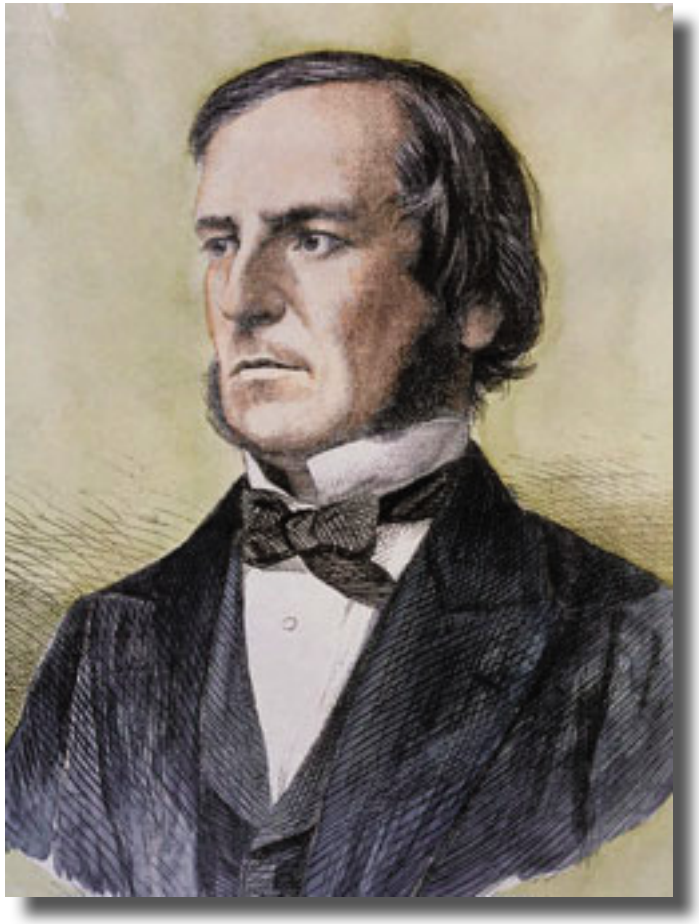
**[Patterson17]**

David A. Patterson & John L. Hennessy
*Computer Organization and Design – The Hardware/Software Interface*
*Appendix A "The Basics of Logic Design"*
ARM edition, Morgan Kaufmann 2017



## *It starts with a thought …*

An Investigation of the Laws of Thought
on Which are Founded the Mathematical
Theories of Logic and Probabilities
by **George Boole**, 1854



George Bool, 1815-1864



## *Boolean Values & Operators*

There are two *values*:

e.g. **True** and **False**. (aka "1" and "0")

Two binary *operators* on expressions *a*, *b*:

$a \vee b$ (aka $a + b$ or "*a* OR *b*" or SUM)

$a \wedge b$ (aka $a \cdot b$ or "*a* AND *b*" or PRODUCT)

One unary *operator* on an expression a:

$\overline{a}$ (aka $\neg a$ or $a'$ or "NOT *a*")

Truth tables:

| $a$ | $b$ | $a \vee b$ | $a \wedge b$ | $\overline{a}$ |
|---|---|---|---|---|
| False | False | False | False | True |
| True | False | True | False | False |
| False | True | True | False | |
| True | True | True | True | |



## *Axiomatic Boolean Algebra (Whitehead 1898)*

| $\vee$-Laws | $\wedge$-Laws | |
|---|---|---|
| $a \vee a = a$ | $a \wedge a = a$ | (redundant) |
| $a \vee b = b \vee a$ | $a \wedge b = b \wedge a$ | (commutative) |
| $(a \vee b) \vee c = a \vee (b \vee c)$ | $(a \wedge b) \wedge c = a \wedge (b \wedge c)$ | (associative) |
| $a \vee (a \wedge b) = a$ | | (absorption) |
| | $a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$ | (distribution) |
| | $a \wedge True = a$ | (identity) |
| | | (constant) |
| $a \vee \overline{a} = True$ | $a \wedge \overline{a} = False$ | (inverse) |
| | | DeMorgan |
| | | (double not) |

*Algebras allow for easier reasoning than truth tables.*



## *Axiomatic Boolean Algebra (Huntington 1904)*

| $\vee$-Laws | $\wedge$-Laws | |
|---|---|---|
| | | (redundant) |
| $a \vee b = b \vee a$ | $a \wedge b = b \wedge a$ | (commutative) |
| | | (associative) |
| | | (absorption) |
| $a \vee (b \wedge c) = (a \vee b) \wedge (a \vee c)$ | $a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$ | (distribution) |
| $a \vee False = a$ | $a \wedge True = a$ | (identity) |
| | | (constant) |
| $a \vee \overline{a} = True$ | $a \wedge \overline{a} = False$ | (inverse) |
| | | DeMorgan |
| | | (double not) |

# *Axiomatic Boolean Algebra*

… many other axiomatic formulations of Boolean algebra exist.



## Redundant Boolean Algebra

*… second nature for a computer scientist!*

| $\vee$-Laws | $\wedge$-Laws | |
|---|---|---|
| $a \vee a = a$ | $a \wedge a = a$ | (redundant) |
| $a \vee b = b \vee a$ | $a \wedge b = b \wedge a$ | (commutative) |
| $(a \vee b) \vee c = a \vee (b \vee c)$ | $(a \wedge b) \wedge c = a \wedge (b \wedge c)$ | (associative) |
| $a \vee (a \wedge b) = a$ | $a \wedge (a \vee b) = a$ | (absorption) |
| $a \vee (b \wedge c) = (a \vee b) \wedge (a \vee c)$ | $a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$ | (distribution) |
| $a \vee \mathit{False} = a$ | $a \wedge \mathit{True} = a$ | (identity) |
| $a \vee \mathit{True} = \mathrm{True}$ | $a \wedge \mathit{False} = \mathrm{False}$ | (constant) |
| $a \vee \overline{a} = \mathit{True}$ | $a \wedge \overline{a} = \mathit{False}$ | (inverse) |
| $\overline{a \vee b} = \overline{a} \wedge \overline{b}$ | $\overline{a \wedge b} = \overline{a} \vee \overline{b}$ | DeMorgan |
| $\overline{\overline{a}} = a$ | | (double not) |



## Common Boolean operators

Commonly used *operators* on expressions $a$, $b$ to define boolean algebras:

$$a \vee b \quad \text{(aka } a + b \text{ or "}a \text{ OR } b\text{" or SUM)}$$
$$a \wedge b \quad \text{(aka } a \cdot b \text{ or "}a \text{ AND } b\text{" or PRODUCT)}$$
$$\overline{a} \quad \text{(aka } \neg a \text{ or } a' \text{or "not } a\text{")}$$

Other handy operators:

$$a \rightarrow b = (\overline{a} \vee b) \quad \text{(aka "}a \text{ IMPLIES } b\text{")}$$
$$(a = b) = (a \wedge b) \vee (\overline{a} \wedge \overline{b}) \quad \text{(aka "}a \text{ EQUALS } b\text{")}$$
$$a \oplus b = (a \wedge \overline{b}) \vee (\overline{a} \wedge b) \quad \text{(aka "}a \text{ EXCLUSIVE-OR } b\text{" or "}a \text{ XOR } b\text{")}$$
$$\overline{a \wedge b} = (\overline{a} \vee \overline{b}) \quad \text{(aka "}a \text{ NOT-AND } b\text{" or "}a \text{ NAND } b\text{")}$$
$$\overline{a \vee b} = (\overline{a} \wedge \overline{b}) \quad \text{(aka "}a \text{ NOT-OR } b\text{" or "}a \text{ NOR } b\text{")}$$

NAND and NOR are the only sole sufficient boolean operators,
i.e. you can reduce any boolean expression to only NAND or only NOR operators.



## All binary Boolean operators

| | Inputs a, b | | | | Function | Name | Sum of products | NAND | Don't cares |
|---|---|---|---|---|---|---|---|---|---|
| a | F | F | T | T | | | | build | |
| b | F | T | F | T | | | | $\wedge$, $\vee$, $\overline{x}$ | |
| q | F | F | F | F | False | Constant FALSE | | | a, b |
| | F | F | F | T | $a \wedge b$ | AND | | $\overline{\overline{a \wedge b} \wedge \overline{a \wedge b}}$ | |
| | F | F | T | F | $\overline{a \rightarrow b}$ | NOT-IMPLICATION | $(a \wedge \overline{b})$ | | |
| | F | F | T | T | a | IDENTITY a | | | b |
| | F | T | F | F | $\overline{b \rightarrow a}$ | NOT-IMPLICATION | $(\overline{a} \wedge b)$ | | |
| | F | T | F | T | b | IDENTITY b | | | a |
| | F | T | T | F | $a \oplus b$ | EXCLUSIVE-OR, XOR | $(a \wedge \overline{b}) \vee (\overline{a} \wedge b)$ | | |
| | F | T | T | T | $a \vee b$ | OR | | $\overline{\overline{a \wedge a} \wedge \overline{b \wedge b}}$ | |
| | T | F | F | F | $\overline{a \vee b}$ | NOT-OR, NOR | $(\overline{a} \wedge \overline{b})$ | | |
| | T | F | F | T | $a = b$ | EQUALITY, EQ | $(a \wedge b) \vee (\overline{a} \wedge \overline{b})$ | | |
| | T | F | T | F | $\overline{b}$ | INVERSE b | | | a |
| | T | F | T | T | $b \rightarrow a$ | IMPLICATION | $a \vee \overline{b}$ | | |
| | T | T | F | F | $\overline{a}$ | INVERSE a | | $\overline{a \wedge a}$ | b |
| | T | T | F | T | $a \rightarrow b$ | IMPLICATION | $\overline{a} \vee b$ | | |
| | T | T | T | F | $\overline{a \wedge b}$ | NOT-AND, NAND | $\overline{a} \vee \overline{b}$ | | |
| | T | T | T | T | True | Constant True | | | a, b |
| | Output q | | | | | | | | |

## *Combinational Logic Functions*

☞ Logic is reducible/equivalent to pure functions: there are no states!

IF the function is combinational then there is only one output for any combination of inputs, e.g. the function can be written out as a truth table:

| a | b | c | Output q |
|---|---|---|---|
| F | F | F | F |
| F | F | T | F |
| F | T | F | T |
| F | T | T | F |
| T | F | F | T |
| T | F | T | T |
| T | T | F | T |
| T | T | T | F |



## *Combinational Logic Functions*

☞ Logic is reducible/equivalent to pure functions: there are no states!

IF the function is combinational then there is only one output for any combination of inputs, e.g. the function can be written out as a truth table:

| a | b | c | Output q | minterms |
|---|---|---|---|---|
| F | F | F | F | |
| F | F | T | F | |
| F | T | F | T | $\overline{a} \wedge b \wedge \overline{c}$ |
| F | T | T | F | |
| T | F | F | T | $a \wedge \overline{b} \wedge \overline{c}$ |
| T | F | T | T | $a \wedge \overline{b} \wedge c$ |
| T | T | F | T | $a \wedge b \wedge \overline{c}$ |
| T | T | T | F | |

Sum of minterms: $q = (\overline{a} \wedge b \wedge \overline{c}) \vee (a \wedge \overline{b} \wedge \overline{c}) \vee (a \wedge \overline{b} \wedge c) \vee (a \wedge b \wedge \overline{c})$



## *Combinational Logic Functions*

☞ Logic is reducible/equivalent to pure functions: there are no states!

IF the function is combinational then there is only one output for any combination of inputs, e.g. the function can be written out as a truth table:

| a | b | c | Output q | minterms | Simplified minterms |
|---|---|---|---|---|---|
| F | F | F | F | | |
| F | F | T | F | | |
| F | T | F | T | $\overline{a} \wedge b \wedge \overline{c}$ | |
| F | T | T | F | | |
| T | F | F | T | $a \wedge \overline{b} \wedge \overline{c}$ | $a \wedge \overline{b}$ |
| T | F | T | T | $a \wedge \overline{b} \wedge c$ | |
| T | T | F | T | $a \wedge b \wedge \overline{c}$ | |
| T | T | T | F | | |

$b \wedge \overline{c}$

Sum of minterms: $q = (\overline{a} \wedge b \wedge \overline{c}) \vee (a \wedge \overline{b} \wedge \overline{c}) \vee (a \wedge \overline{b} \wedge c) \vee (a \wedge b \wedge \overline{c})$

Sum of simplified minterms: $q = (a \wedge \overline{b}) \vee (b \wedge \overline{c})$

Simplifications can be done by (automated) algebraic transformations, Karnaugh maps or others



## *Combinational Logic Functions*

☞ Logic is reducible/equivalent to pure functions: there are no states!

IF the function is combinational then there is only one output for any combination of inputs, e.g. the function can be written out as a truth table:

| a | b | c | Output q | minterms | Simplified minterms | |
|---|---|---|---|---|---|---|
| F | F | F | F | | | |
| F | F | T | F | | | |
| F | T | F | T | $\overline{a} \wedge b \wedge \overline{c}$ | | $b \wedge \overline{c}$ |
| F | T | T | F | | | |
| T | F | F | T | $a \wedge \overline{b} \wedge \overline{c}$ | $a \wedge \overline{b}$ | |
| T | F | T | T | $a \wedge \overline{b} \wedge c$ | | |
| T | T | F | T | $a \wedge b \wedge \overline{c}$ | | |
| T | T | T | F | | | |

*Every* combinational function can be written as a **sum of products**!

Sum of minterms: $q = (\overline{a} \wedge b \wedge \overline{c}) \vee (a \wedge \overline{b} \wedge \overline{c}) \vee (a \wedge \overline{b} \wedge c) \vee (a \wedge b \wedge \overline{c})$

Sum of simplified minterms: $q = (a \wedge \overline{b}) \vee (b \wedge \overline{c})$

Simplifications can be done by (automated) algebraic transformations, Karnaugh maps or others

## *Combinational Logic Functions*

☞ Logic is reducible/equivalent to pure functions: there are no states!

IF the function is combinational then there is only one output for any combination of inputs, e.g. the function can be written out as a truth table:

| a | b | c | Output q |
|---|---|---|---|
| F | F | F | F |
| F | F | T | F |
| F | T | F | T |
| F | T | T | F |
| T | F | F | T |
| T | F | T | T |
| T | T | F | T |
| T | T | T | F |



## *Combinational Logic Functions*

☞ Logic is reducible/equivalent to pure functions: there are no states!

IF the function is combinational then there is only one output for any combination of inputs, e.g. the function can be written out as a truth table:

| a | b | c | Output q | maxterms |
|---|---|---|---|---|
| F | F | F | F | $a \vee b \vee c$ |
| F | F | T | F | $a \vee b \vee \overline{c}$ |
| F | T | F | T | |
| F | T | T | F | $a \vee \overline{b} \vee \overline{c}$ |
| T | F | F | T | |
| T | F | T | T | |
| T | T | F | T | |
| T | T | T | F | $\overline{a} \vee \overline{b} \vee \overline{c}$ |

maxterms product $q = (a \vee b \vee c) \wedge (a \vee b \vee \overline{c}) \wedge (a \vee \overline{b} \vee \overline{c}) \wedge (\overline{a} \vee \overline{b} \vee \overline{c})$



## *Combinational Logic Functions*

☞ Logic is reducible/equivalent to pure functions: there are no states!

IF the function is combinational then there is only one output for any combination of inputs, e.g. the function can be written out as a truth table:

| a | b | c | Output q | maxterms | Simplified maxterms |
|---|---|---|---|---|---|
| F | F | F | F | $a \vee b \vee c$ | $a \vee b$ |
| F | F | T | F | $a \vee b \vee \overline{c}$ | |
| F | T | F | T | | |
| F | T | T | F | $a \vee \overline{b} \vee \overline{c}$ | $\overline{b} \vee \overline{c}$ |
| T | F | F | T | | |
| T | F | T | T | | |
| T | T | F | T | | |
| T | T | T | F | $\overline{a} \vee \overline{b} \vee \overline{c}$ | |

maxterms product $q = (a \vee b \vee c) \wedge (a \vee b \vee \overline{c}) \wedge (a \vee \overline{b} \vee \overline{c}) \wedge (\overline{a} \vee \overline{b} \vee \overline{c})$

simplified maxterms product $q = (a \vee b) \wedge (\overline{b} \vee \overline{c})$

Simplifications can be done by (automated) algebraic transformations, Karnaugh maps or others



## *Combinational Logic Functions*

☞ Logic is reducible/equivalent to pure functions: there are no states!

IF the function is combinational then there is only one output for any combination of inputs, e.g. the function can be written out as a truth table:

| a | b | c | Output q | maxterms | Simplified maxterms |
|---|---|---|---|---|---|
| F | F | F | F | $a \vee b \vee c$ | $a \vee b$ |
| F | F | T | F | $a \vee b \vee \overline{c}$ | |
| F | T | F | T | | |
| F | T | T | F | $a \vee \overline{b} \vee \overline{c}$ | $\overline{b} \vee \overline{c}$ |
| T | F | F | T | | |
| T | F | T | T | | |
| T | T | F | T | | |
| T | T | T | F | $\overline{a} \vee \overline{b} \vee \overline{c}$ | |

*Every* combinational function can be written as a **product of sums**!

maxterms product $q = (a \vee b \vee c) \wedge (a \vee b \vee \overline{c}) \wedge (a \vee \overline{b} \vee \overline{c}) \wedge (\overline{a} \vee \overline{b} \vee \overline{c})$

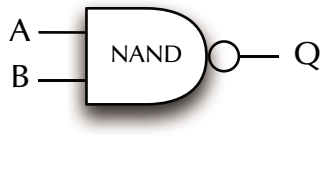
simplified maxterms product $q = (a \vee b) \wedge (\overline{b} \vee \overline{c})$

Simplifications can be done by (automated) algebraic transformations, Karnaugh maps or others

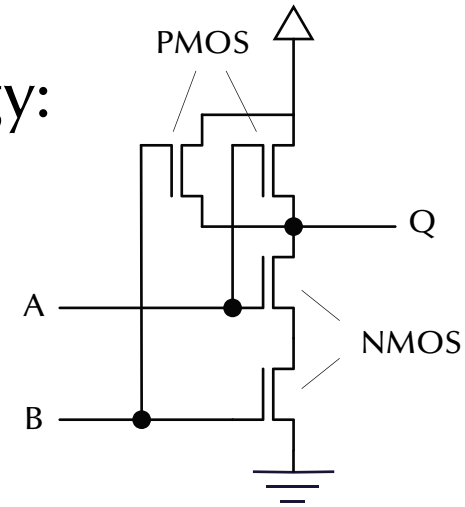
## *Digital Electronics*

Symbolic: $Q = \overline{A \wedge B}$

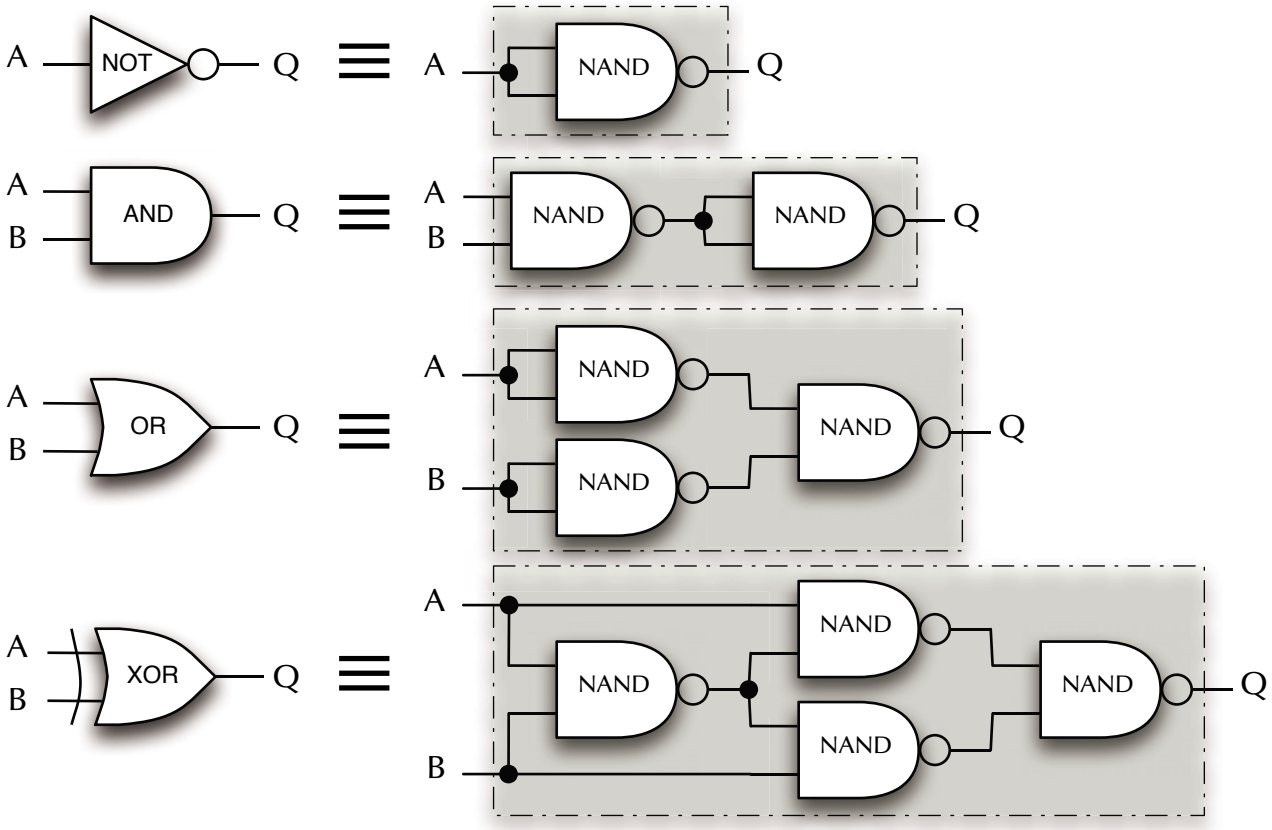
Diagram:




Technology:




Elementary logic gate symbols:

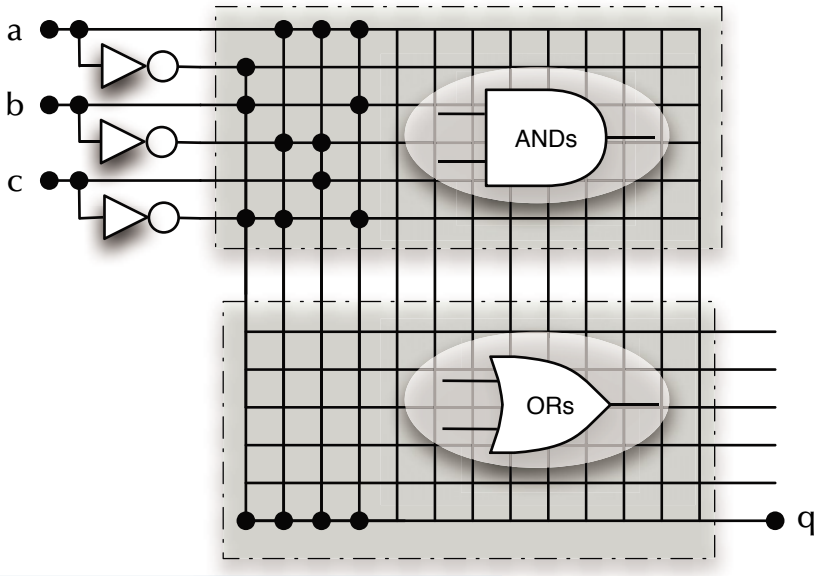


## Combinational Logic Functions

☞ The logic equivalent to pure functions: there are no states!

IF the function is combinational then there is only one output for any combination of inputs, e.g. the function can be written out as a truth table:

| a | b | c | Output q | Minterms | Simplified minterms | |
|---|---|---|---|---|---|---|
| F | F | F | F | | | |
| F | F | T | F | | | |
| F | T | F | T | $\overline{a} \wedge b \wedge \overline{c}$ | | $b \wedge \overline{c}$ |
| F | T | T | F | | | |
| T | F | F | T | $a \wedge \overline{b} \wedge \overline{c}$ | $a \wedge \overline{b}$ | |
| T | F | T | T | $a \wedge \overline{b} \wedge c$ | | |
| T | T | F | T | $a \wedge b \wedge \overline{c}$ | | |
| T | T | T | F | | | |




Sum of minterms: $q = (\overline{a} \wedge b \wedge \overline{c}) \vee (a \wedge \overline{b} \wedge \overline{c}) \vee (a \wedge \overline{b} \wedge c) \vee (a \wedge b \wedge \overline{c})$

Sum of simplified minterms: $q = (a \wedge \overline{b}) \vee (b \wedge \overline{c})$

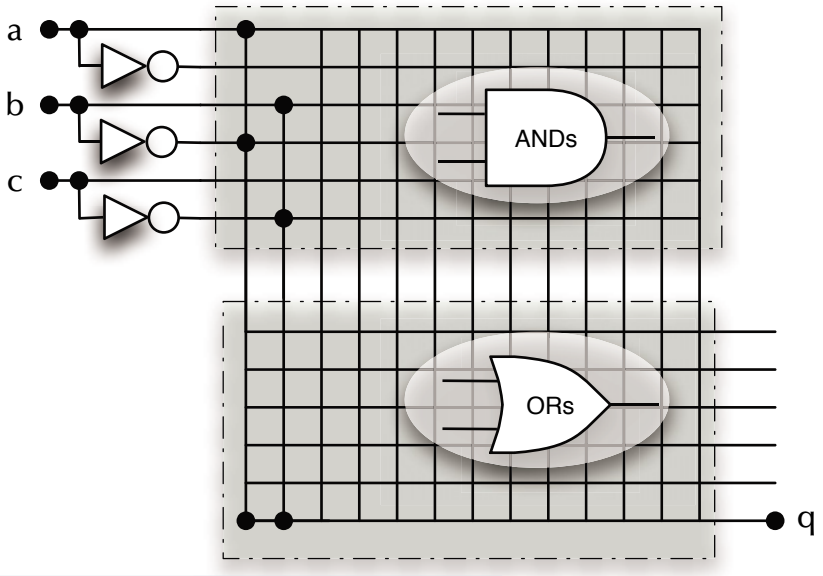
Simplifications can be done by (automated) algebraic transformations, Karnaugh maps or others

## Combinational Logic Functions

☞ The logic equivalent to pure functions: there are no states!

IF the function is combinational then there is only one output for any combination of inputs, e.g. the function can be written out as a truth table:

| a | b | c | Output q | Minterms | Simplified minterms | |
|---|---|---|---|---|---|---|
| F | F | F | F | | | |
| F | F | T | F | | | |
| F | T | F | T | $\overline{a} \wedge b \wedge \overline{c}$ | | |
| F | T | T | F | | | |
| T | F | F | T | $a \wedge \overline{b} \wedge \overline{c}$ | $a \wedge \overline{b}$ | $b \wedge \overline{c}$ |
| T | F | T | T | $a \wedge \overline{b} \wedge c$ | | |
| T | T | F | T | $a \wedge b \wedge \overline{c}$ | | |
| T | T | T | F | | | |




Sum of minterms: $q = (\overline{a} \wedge b \wedge \overline{c}) \vee (a \wedge \overline{b} \wedge \overline{c}) \vee (a \wedge \overline{b} \wedge c) \vee (a \wedge b \wedge \overline{c})$

Sum of simplified minterms: $q = (a \wedge \overline{b}) \vee (b \wedge \overline{c})$

Simplifications can be done by (automated) algebraic transformations, Karnaugh maps or others



## Combinational Logic Functions

☞ The logic equivalent to pure functions: there are no states!

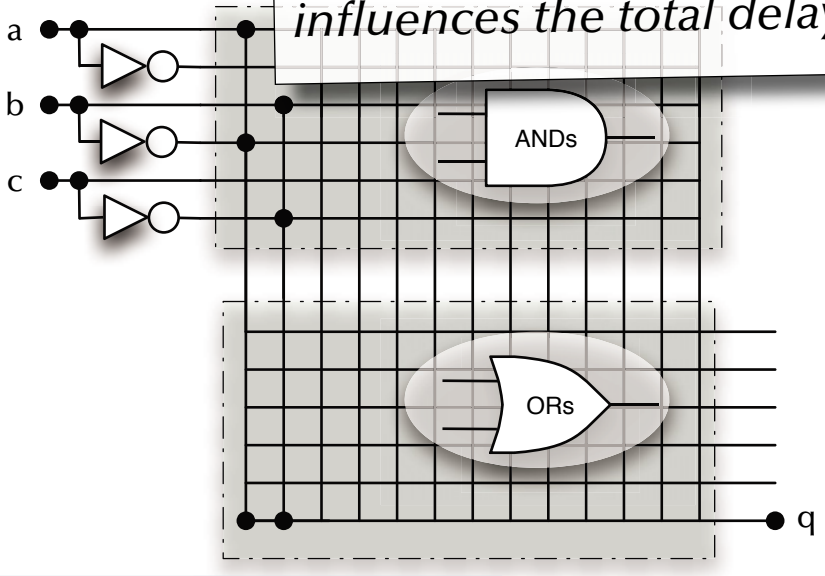
IF the function is combinational then there is only one output for any combination of inputs,
e.g. the function can be written out as a truth table:

| a | b | c | Output q | Minterms | Simplified minterms | |
|---|---|---|---|---|---|---|
| F | F | F | F | | | |
| F | F | T | F | | | |
| F | T | F | T | $\overline{a} \wedge b \wedge \overline{c}$ | | $b \wedge \overline{c}$ |
| F | T | T | F | | | |
| T | F | F | T | $a \wedge \overline{b} \wedge \overline{c}$ | $a \wedge \overline{b}$ | |
| T | F | T | T | $a \wedge \overline{b} \wedge c$ | | |
| T | T | F | T | $a \wedge b \wedge \overline{c}$ | | |
| T | T | T | F | | | |

Sum of minterms: $q = (\overline{a} \wedge b \wedge \overline{c}) \vee (a \wedge \overline{b} \wedge \overline{c}) \vee (a \wedge \overline{b} \wedge c) \vee (a \wedge b \wedge \overline{c})$

Sum of simplified minterms: $q = (a \wedge \overline{b}) \vee (b \wedge \overline{c})$

*The number of terms ("fan-in" for the gates) influences the total delay*



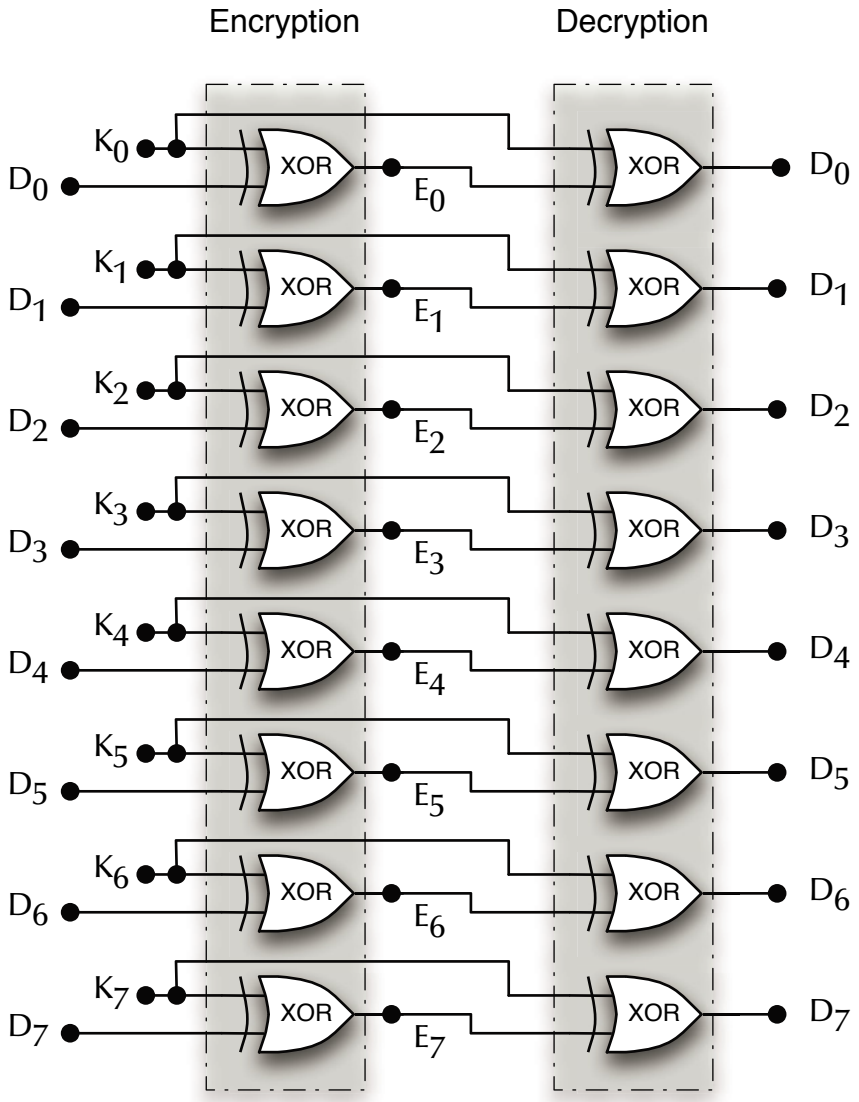

Simplifications can be done by (automated) algebraic transformations, Karnaugh maps or others

## *Processing Data*

Encrypting a bit vector (whatever it represents) with a secret key:

Assuming the key is random and not used for anything else:

- ☞ This is surprisingly secure
- ☞ … and extremely fast!





## *Bit Vectors*

Groups of bits could represent:

States, enumeration values, arrays of Booleans, numbers, etc. pp.
… or any grouping or combination of the above ☞ **Algebraic Types**

The form of encoding could be chosen to optimize for:

- **Performance** ☞ e.g. minimal decoding effort
- **Redundancy** / error detection ☞ e.g. large Hamming distance
- **Safe transitions** ☞ e.g. Gray codes
- **Physical mapping** ☞ e.g. maps on existing hardware interfaces
- **Compactness** ☞ e.g. holds the maximal number of values per memory cell



## Encoding

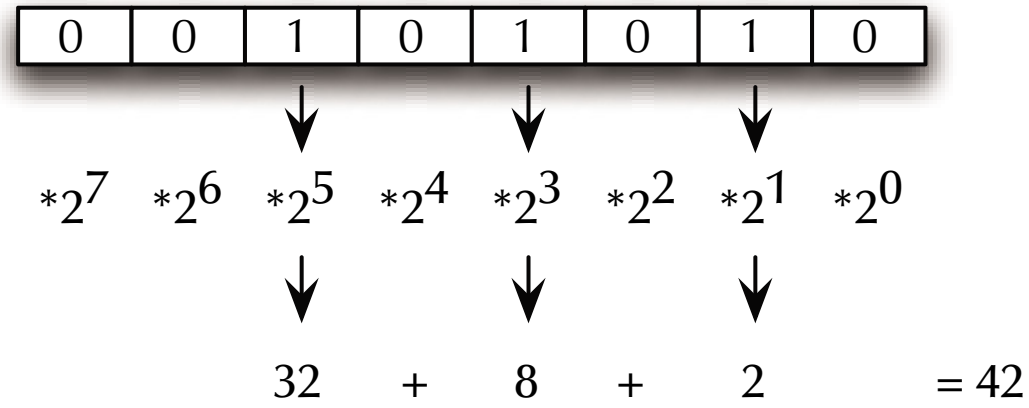
Assuming a type can have 7 different values, many forms of encoding are possible:

| Enumeration type | | Encoding | | | | | |
|---|---|---|---|---|---|---|---|
| | | | | 1-bit error detecting | 1-bit error correcting | | |
| Index | Value | Single bit | Gray code | Even parity | Hamming (7,4) | Hamming (3,1) | Binary |
| 1 | Secured | 0000001 | 000 | 0000 | 0000000 | 000000000 | 000 |
| 2 | Taxi | 0000010 | 001 | 0011 | 1110000 | 000000111 | 001 |
| 3 | Take-off | 0000100 | 011 | 0101 | 1001100 | 000111000 | 010 |
| 4 | Cruising | 0001000 | 010 | 0110 | 0111100 | 000111111 | 011 |
| 5 | Gliding | 0010000 | 110 | 1001 | 0101010 | 111000000 | 100 |
| 6 | Approach | 0100000 | 111 | 1010 | 1011010 | 111000111 | 101 |
| 7 | Landing | 1000000 | 101 | 1100 | 1100110 | 111111000 | 110 |

☞ VHDL or Verilog gives you full control over the encoding.

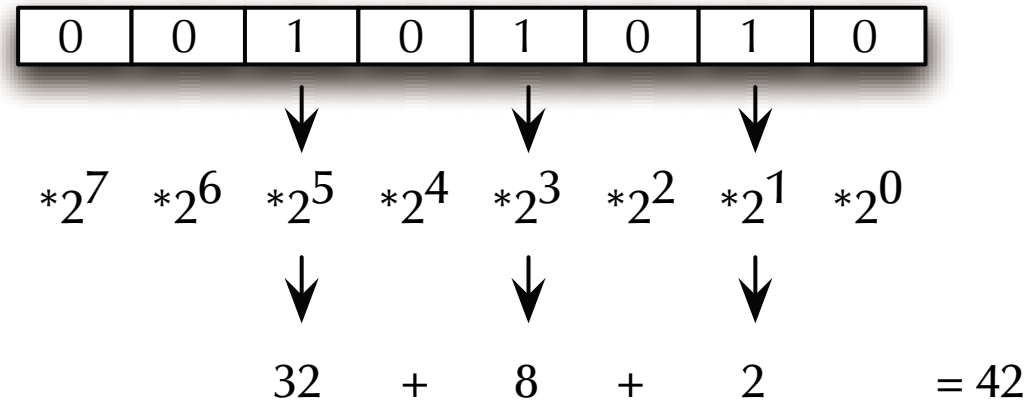
## *Binary encoding*

☞ Encoding of choice if compactness is essential or you need to add values.



## Binary encoding

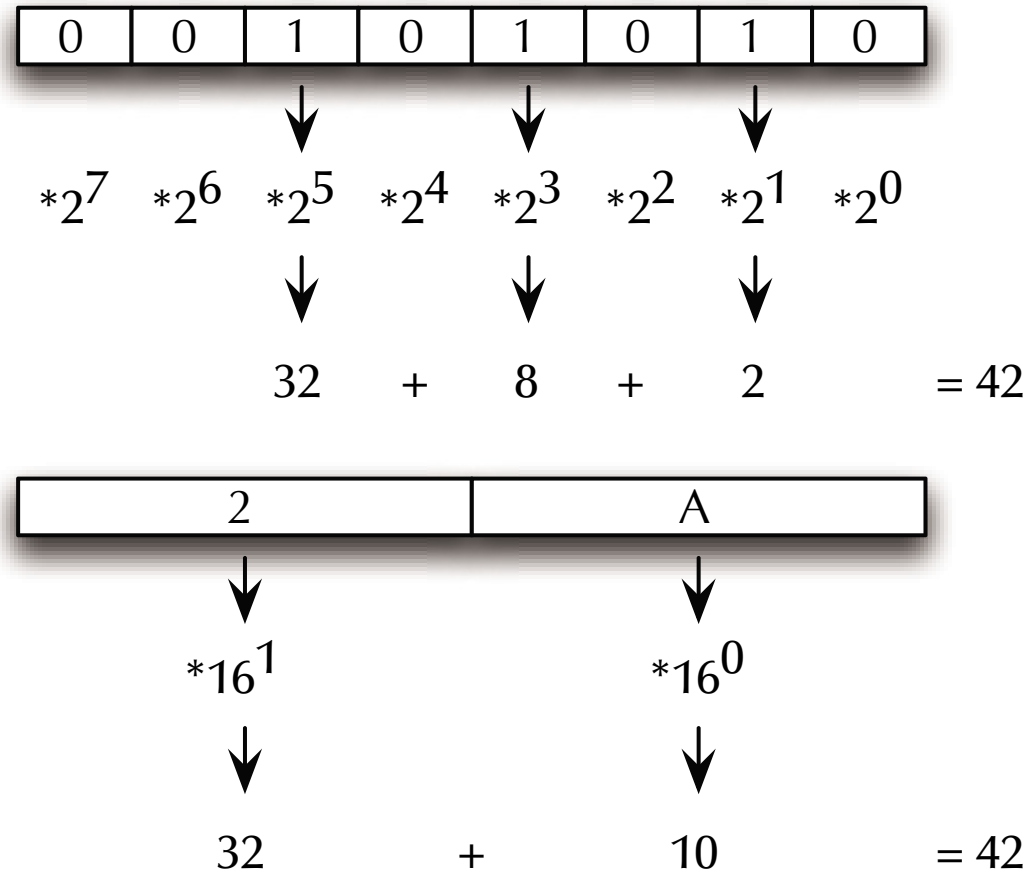
☞ Encoding of choice if compactness is essential or you need to add values.




| Decimal | | Binary | | Hexadecimal |
|---|---|---|---|---|
| 0 | = | 0 0 0 0 | = | 0 |
| 1 | = | 0 0 0 1 | = | 1 |
| 2 | = | 0 0 1 0 | = | 2 |
| 3 | = | 0 0 1 1 | = | 3 |
| 4 | = | 0 1 0 0 | = | 4 |
| 5 | = | 0 1 0 1 | = | 5 |
| 6 | = | 0 1 1 0 | = | 6 |
| 7 | = | 0 1 1 1 | = | 7 |
| 8 | = | 1 0 0 0 | = | 8 |
| 9 | = | 1 0 0 1 | = | 9 |
| 10 | = | 1 0 1 0 | = | A |
| 11 | = | 1 0 1 1 | = | B |
| 12 | = | 1 1 0 0 | = | C |
| 13 | = | 1 1 0 1 | = | D |
| 14 | = | 1 1 1 0 | = | E |
| 15 | = | 1 1 1 1 | = | F |

## *Binary encoding*

☞ Encoding of choice if compactness is essential or you need to add values.




| Decimal | | Binary | | Hexadecimal |
|---|---|---|---|---|
| 0 | = | 0 0 0 0 | = | 0 |
| 1 | = | 0 0 0 1 | = | 1 |
| 2 | = | 0 0 1 0 | = | 2 |
| 3 | = | 0 0 1 1 | = | 3 |
| 4 | = | 0 1 0 0 | = | 4 |
| 5 | = | 0 1 0 1 | = | 5 |
| 6 | = | 0 1 1 0 | = | 6 |
| 7 | = | 0 1 1 1 | = | 7 |
| 8 | = | 1 0 0 0 | = | 8 |
| 9 | = | 1 0 0 1 | = | 9 |
| 10 | = | 1 0 1 0 | = | A |
| 11 | = | 1 0 1 1 | = | B |
| 12 | = | 1 1 0 0 | = | C |
| 13 | = | 1 1 0 1 | = | D |
| 14 | = | 1 1 1 0 | = | E |
| 15 | = | 1 1 1 1 | = | F |

## Half Adder

| $A$ | $B$ | $S$ | $C$ | $S$ minterms | $C$ minterms |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | | |
| 0 | 1 | 1 | 0 | $\overline{A} \wedge B$ | |
| 1 | 0 | 1 | 0 | $A \wedge \overline{B}$ | |
| 1 | 1 | 0 | 1 | | $A \wedge B$ |



## Half Adder

| $A$ | $B$ | $S$ | $C$ | $S$ minterms | $C$ minterms |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | | |
| 0 | 1 | 1 | 0 | $\overline{A} \wedge B$ | |
| 1 | 0 | 1 | 0 | $A \wedge \overline{B}$ | |
| 1 | 1 | 0 | 1 | | $A \wedge B$ |

☞ $S = (A \wedge \overline{B}) \vee (\overline{A} \wedge B)$

☞ $C = A \wedge B$

## Half Adder

| $A$ | $B$ | $S$ | $C$ | $S$ minterms | $C$ minterms |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | | |
| 0 | 1 | 1 | 0 | $\overline{A} \wedge B$ | |
| 1 | 0 | 1 | 0 | $A \wedge \overline{B}$ | |
| 1 | 1 | 0 | 1 | | $A \wedge B$ |

☞ $S = \left(A \wedge \overline{B}\right) \vee \left(\overline{A} \wedge B\right) = A \oplus B$

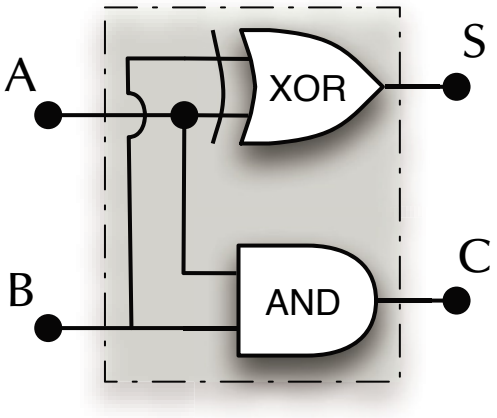
☞ $C = A \wedge B$

## Half Adder

| $A$ | $B$ | $S$ | $C$ | $S$ minterms | $C$ minterms |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | | |
| 0 | 1 | 1 | 0 | $\overline{A} \wedge B$ | |
| 1 | 0 | 1 | 0 | $A \wedge \overline{B}$ | |
| 1 | 1 | 0 | 1 | | $A \wedge B$ |

☞ $S = \left(A \wedge \overline{B}\right) \vee \left(\overline{A} \wedge B\right) = A \oplus B$

☞ $C = A \wedge B$



## Full Adder

| $A_i$ | $B_i$ | $C_{i-1}$ | $S_i$ | $C_i$ | | |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | | |
| 0 | 1 | 0 | 1 | 0 | | |
| 1 | 0 | 0 | 1 | 0 | | |
| 1 | 1 | 0 | 0 | 1 | | |
| 0 | 0 | 1 | 1 | 0 | | |
| 0 | 1 | 1 | 0 | 1 | | |
| 1 | 0 | 1 | 0 | 1 | | |
| 1 | 1 | 1 | 1 | 1 | | |

## Full Adder

| $A_i$ | $B_i$ | $C_{i-1}$ | $S_i$ | $C_i$ | $S_i$ minterms | $C_i$ minterms |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | | |
| 0 | 1 | 0 | 1 | 0 | $\overline{A_i} \wedge B_i \wedge \overline{C_{i-1}}$ | |
| 1 | 0 | 0 | 1 | 0 | $A_i \wedge \overline{B_i} \wedge \overline{C_{i-1}}$ | |
| 1 | 1 | 0 | 0 | 1 | | $A_i \wedge B_i \wedge \overline{C_{i-1}}$ |
| 0 | 0 | 1 | 1 | 0 | $\overline{A_i} \wedge \overline{B_i} \wedge C_{i-1}$ | |
| 0 | 1 | 1 | 0 | 1 | | $\overline{A_i} \wedge B_i \wedge C_{i-1}$ |
| 1 | 0 | 1 | 0 | 1 | | $A_i \wedge \overline{B_i} \wedge C_{i-1}$ |
| 1 | 1 | 1 | 1 | 1 | $A_i \wedge B_i \wedge C_{i-1}$ | $A_i \wedge B_i \wedge C_{i-1}$ |

☞ $S_i = \left(A_i \wedge \overline{B_i} \wedge \overline{C_{i-1}}\right) \vee \left(\overline{A_i} \wedge B_i \wedge \overline{C_{i-1}}\right) \vee \left(\overline{A_i} \wedge \overline{B_i} \wedge C_{i-1}\right) \vee \left(A_i \wedge B_i \wedge C_{i-1}\right)$



## Full Adder

| $A_i$ | $B_i$ | $C_{i-1}$ | $S_i$ | $C_i$ | $S_i$ minterms | $C_i$ minterms |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | | |
| 0 | 1 | 0 | 1 | 0 | $\overline{A_i} \wedge B_i \wedge \overline{C_{i-1}}$ | |
| 1 | 0 | 0 | 1 | 0 | $A_i \wedge \overline{B_i} \wedge \overline{C_{i-1}}$ | |
| 1 | 1 | 0 | 0 | 1 | | $A_i \wedge B_i \wedge \overline{C_{i-1}}$ |
| 0 | 0 | 1 | 1 | 0 | $\overline{A_i} \wedge \overline{B_i} \wedge C_{i-1}$ | |
| 0 | 1 | 1 | 0 | 1 | | $\overline{A_i} \wedge B_i \wedge C_{i-1}$ |
| 1 | 0 | 1 | 0 | 1 | | $A_i \wedge \overline{B_i} \wedge C_{i-1}$ |
| 1 | 1 | 1 | 1 | 1 | $A_i \wedge B_i \wedge C_{i-1}$ | $A_i \wedge B_i \wedge C_{i-1}$ |

☞ $$S_i = \left(A_i \wedge \overline{B_i} \wedge \overline{C_{i-1}}\right) \vee \left(\overline{A_i} \wedge B_i \wedge \overline{C_{i-1}}\right) \vee \left(\overline{A_i} \wedge \overline{B_i} \wedge C_{i-1}\right) \vee \left(A_i \wedge B_i \wedge C_{i-1}\right)$$
$$= \left(\left(\left(A_i \wedge \overline{B_i}\right) \vee \left(\overline{A_i} \wedge B_i\right)\right) \wedge \overline{C_{i-1}}\right) \vee \left(\left(\left(\overline{A_i} \wedge \overline{B_i}\right) \vee \left(A_i \wedge B_i\right)\right) \wedge C_{i-1}\right)$$
$$= \left(\left(A_i \oplus B_i\right) \wedge \overline{C_{i-1}}\right) \vee \left(\left(A_i = B_i\right) \wedge C_{i-1}\right) = \left(\left(A_i \oplus B_i\right) \wedge \overline{C_{i-1}}\right) \vee \left(\left(\overline{A_i \oplus B_i}\right) \wedge C_{i-1}\right)$$
$$= \left(A_i \oplus B_i\right) \oplus C_{i-1}$$

## Full Adder

| $A_i$ | $B_i$ | $C_{i-1}$ | $S_i$ | $C_i$ | $S_i$ minterms | $C_i$ minterms |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | | |
| 0 | 1 | 0 | 1 | 0 | $\overline{A_i} \wedge B_i \wedge \overline{C_{i-1}}$ | |
| 1 | 0 | 0 | 1 | 0 | $A_i \wedge \overline{B_i} \wedge \overline{C_{i-1}}$ | |
| 1 | 1 | 0 | 0 | 1 | | $A_i \wedge B_i \wedge \overline{C_{i-1}}$ |
| 0 | 0 | 1 | 1 | 0 | $\overline{A_i} \wedge \overline{B_i} \wedge C_{i-1}$ | |
| 0 | 1 | 1 | 0 | 1 | | $\overline{A_i} \wedge B_i \wedge C_{i-1}$ |
| 1 | 0 | 1 | 0 | 1 | | $A_i \wedge \overline{B_i} \wedge C_{i-1}$ |
| 1 | 1 | 1 | 1 | 1 | $A_i \wedge B_i \wedge C_{i-1}$ | $A_i \wedge B_i \wedge C_{i-1}$ |

☞ $S_i = (A_i \oplus B_i) \oplus C_{i-1}$

☞ $C_i = \left(A_i \wedge B_i \wedge \overline{C_{i-1}}\right) \vee \left(\overline{A_i} \wedge B_i \wedge C_{i-1}\right) \vee \left(A_i \wedge \overline{B_i} \wedge C_{i-1}\right) \vee \left(A_i \wedge B_i \wedge C_{i-1}\right)$

## Full Adder

| $A_i$ | $B_i$ | $C_{i-1}$ | $S_i$ | $C_i$ | $S_i$ minterms | $C_i$ minterms |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | | |
| 0 | 1 | 0 | 1 | 0 | $\overline{A_i} \wedge B_i \wedge \overline{C_{i-1}}$ | |
| 1 | 0 | 0 | 1 | 0 | $A_i \wedge \overline{B_i} \wedge \overline{C_{i-1}}$ | |
| 1 | 1 | 0 | 0 | 1 | | $A_i \wedge B_i \wedge \overline{C_{i-1}}$ |
| 0 | 0 | 1 | 1 | 0 | $\overline{A_i} \wedge \overline{B_i} \wedge C_{i-1}$ | |
| 0 | 1 | 1 | 0 | 1 | | $\overline{A_i} \wedge B_i \wedge C_{i-1}$ |
| 1 | 0 | 1 | 0 | 1 | | $A_i \wedge \overline{B_i} \wedge C_{i-1}$ |
| 1 | 1 | 1 | 1 | 1 | $A_i \wedge B_i \wedge C_{i-1}$ | $A_i \wedge B_i \wedge C_{i-1}$ |

☞ $S_i = (A_i \oplus B_i) \oplus C_{i-1}$

☞ $C_i = \left(A_i \wedge B_i \wedge \overline{C_{i-1}}\right) \vee \left(\overline{A_i} \wedge B_i \wedge C_{i-1}\right) \vee \left(A_i \wedge \overline{B_i} \wedge C_{i-1}\right) \vee \left(A_i \wedge B_i \wedge C_{i-1}\right)$

$= \left(A_i \wedge B_i \wedge \overline{C_{i-1}}\right) \vee \left(A_i \wedge B_i \wedge C_{i-1}\right) \vee \left(\overline{A_i} \wedge B_i \wedge C_{i-1}\right) \vee \left(A_i \wedge \overline{B_i} \wedge C_{i-1}\right)$

$= \left(A_i \wedge B_i\right) \vee \left(\left(\left(\overline{A_i} \wedge B_i\right) \vee \left(A_i \wedge \overline{B_i}\right)\right) \wedge C_{i-1}\right)$

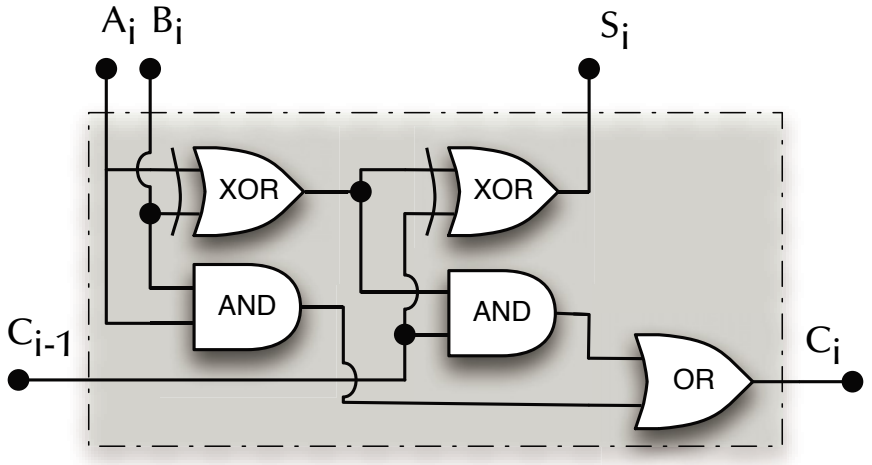
$= \left(A_i \wedge B_i\right) \vee \left(\left(A_i \oplus B_i\right) \wedge C_{i-1}\right)$

## Full Adder

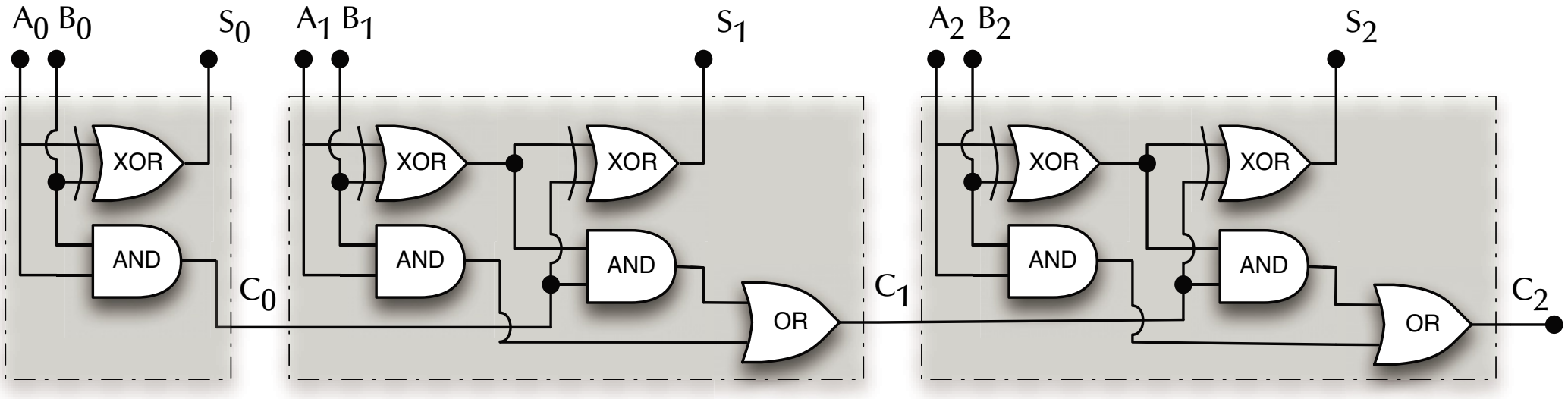
| $A_i$ | $B_i$ | $C_{i-1}$ | $S_i$ | $C_i$ | $S_i$ minterms | $C_i$ minterms |
|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | | |
| 0 | 1 | 0 | 1 | 0 | $\overline{A_i} \wedge B_i \wedge \overline{C_{i-1}}$ | |
| 1 | 0 | 0 | 1 | 0 | $A_i \wedge \overline{B_i} \wedge \overline{C_{i-1}}$ | |
| 1 | 1 | 0 | 0 | 1 | | $A_i \wedge B_i \wedge \overline{C_{i-1}}$ |
| 0 | 0 | 1 | 1 | 0 | $\overline{A_i} \wedge \overline{B_i} \wedge C_{i-1}$ | |
| 0 | 1 | 1 | 0 | 1 | | $\overline{A_i} \wedge B_i \wedge C_{i-1}$ |
| 1 | 0 | 1 | 0 | 1 | | $A_i \wedge \overline{B_i} \wedge C_{i-1}$ |
| 1 | 1 | 1 | 1 | 1 | $A_i \wedge B_i \wedge C_{i-1}$ | $A_i \wedge B_i \wedge C_{i-1}$ |

☞ $S_i = (A_i \oplus B_i) \oplus C_{i-1}$

☞ $C_i = (A_i \wedge B_i) \vee ((A_i \oplus B_i) \wedge C_{i-1})$

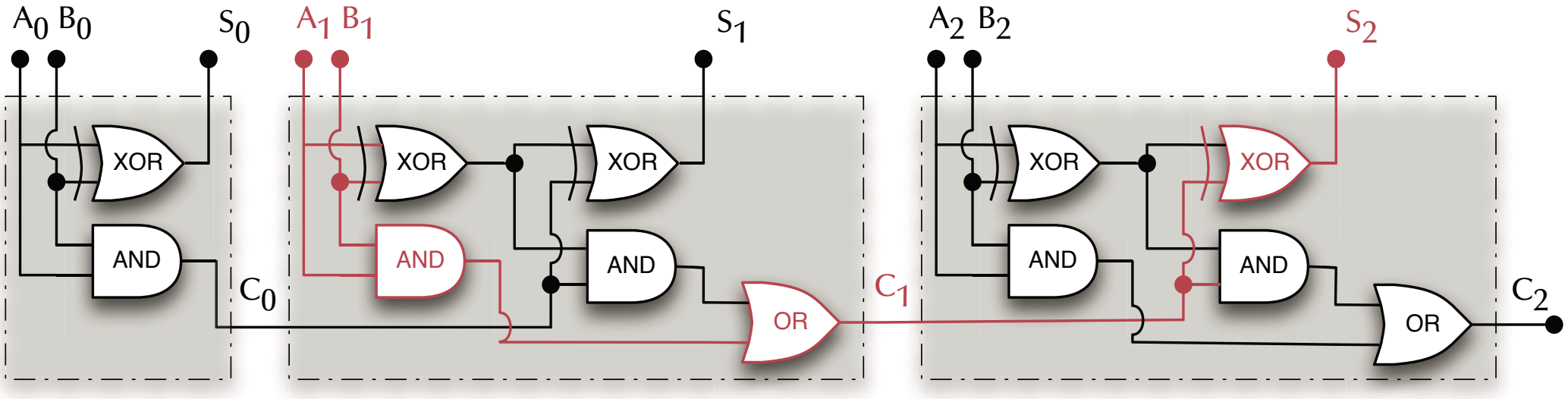


## Ripple Carry Adder



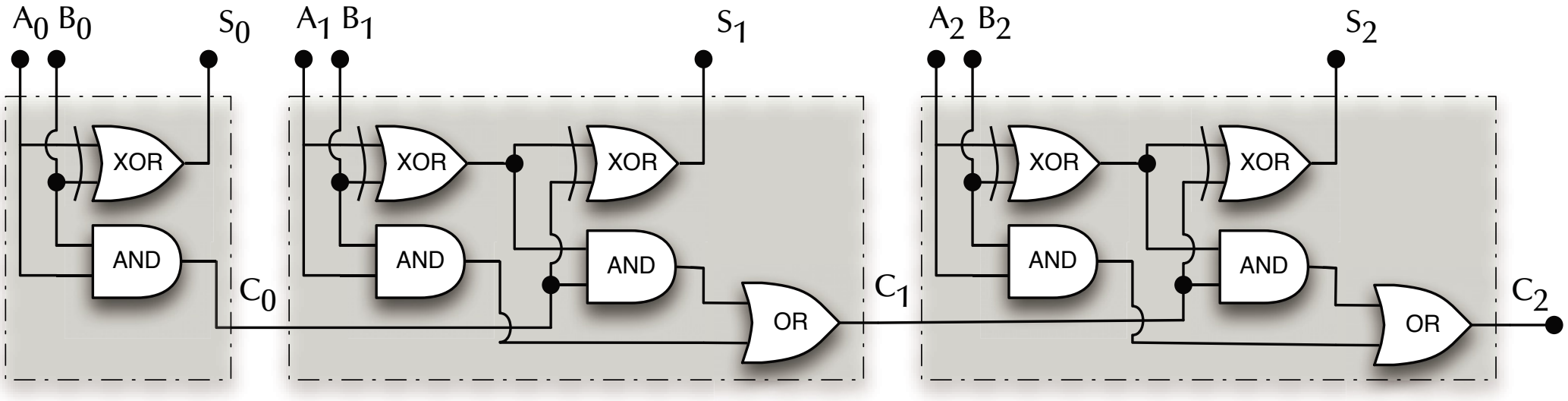

$$2 + 2 = 4\ ?$$

## Ripple Carry Adder




$$2 + 2 = 4\ !$$

## Ripple Carry Adder



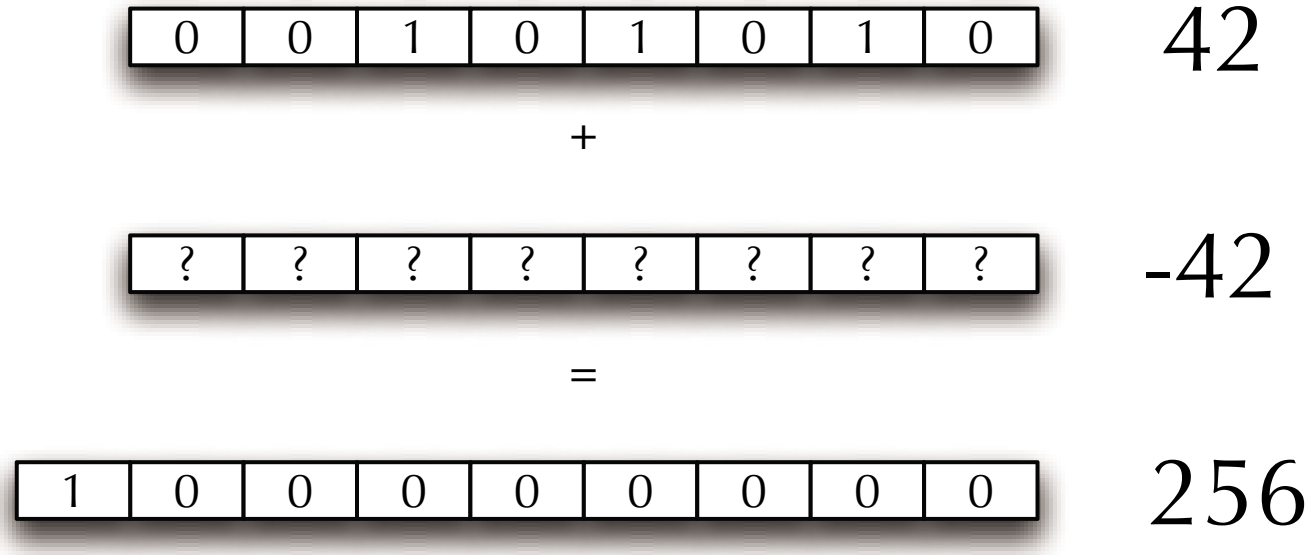

$2 - 1 = 1$ ?

## *Radix complements*

☞ Can we define negative numbers such that our adder still works?

$$x - x = 0$$

Or: what can you add to 42 in an 8 bit binary representation such that the result will be $2^8$ (and hence 0 in 8 bits)?





## *Radix complements*

☞ Can we define negative numbers such that our adder still works?

$$x - x = 0$$

Or: what can you add to 42 in an 8 bit binary representation such that the result will be $2^8$ (and hence 0 in 8 bits)?

| 0 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 42 |
|---|---|---|---|---|---|---|---|---|
| + | | | | | | | | |
| 1 | 1 | 0 | 1 | 0 | 1 | 1 | 0 | -42 |
| = | | | | | | | | |
| 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 256 |

## *Radix complements*

☞ Can we define negative numbers such that our adder still works?

$$x - x = 0$$

Or: what can you add to 42 in an 8 bit binary representation such that the result will be $2^8$ (and hence 0 in 8 bits)?

*"Invert all bits and add 1"*

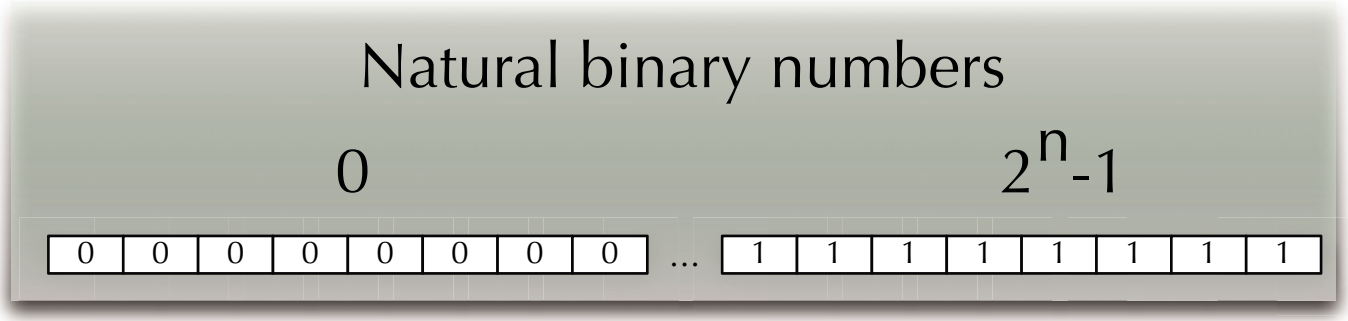
**2's-complement**

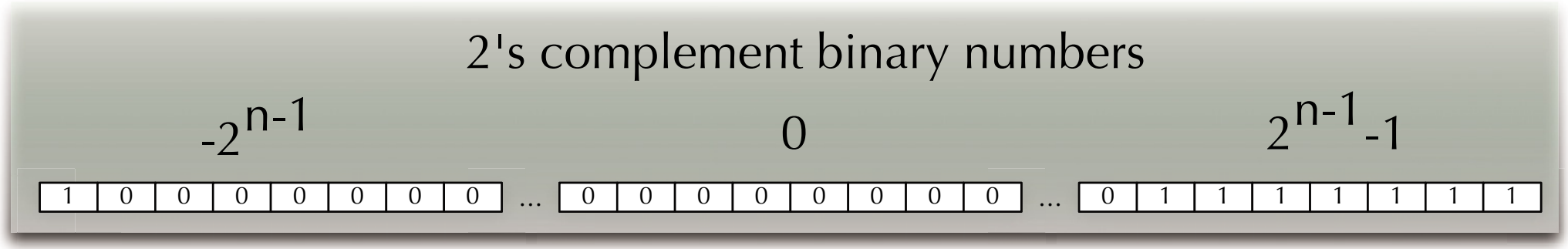
(as the radix/base is 2)

| 0 | 0 | 1 | 0 | 1 | 0 | 1 | 0 | 42 |
|---|---|---|---|---|---|---|---|---|
| + | | | | | | | | |
| 1 | 1 | 0 | 1 | 0 | 1 | 1 | 0 | -42 |
| = | | | | | | | | |
| 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 256 |

## *2's complements*

The 2's complement encoding interprets
the natural binary range $2^{n-1} \ldots 2^n - 1$
as negative numbers $-2^{n-1} \ldots -1$

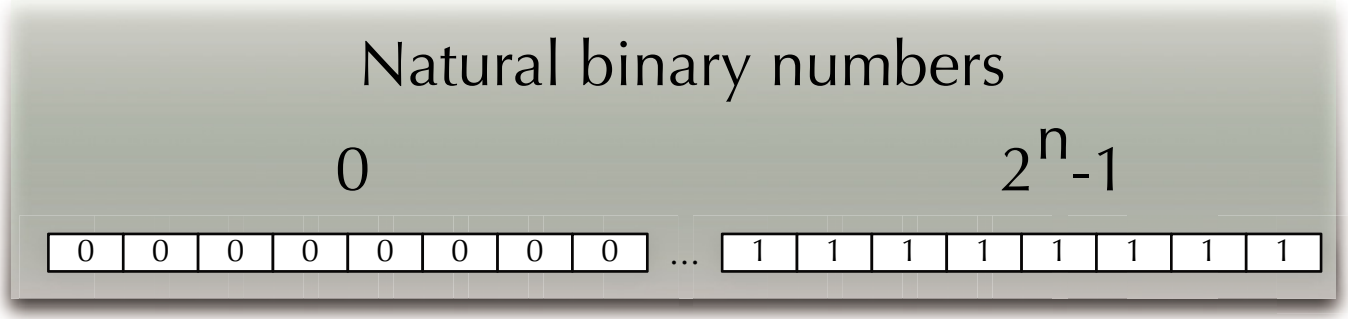


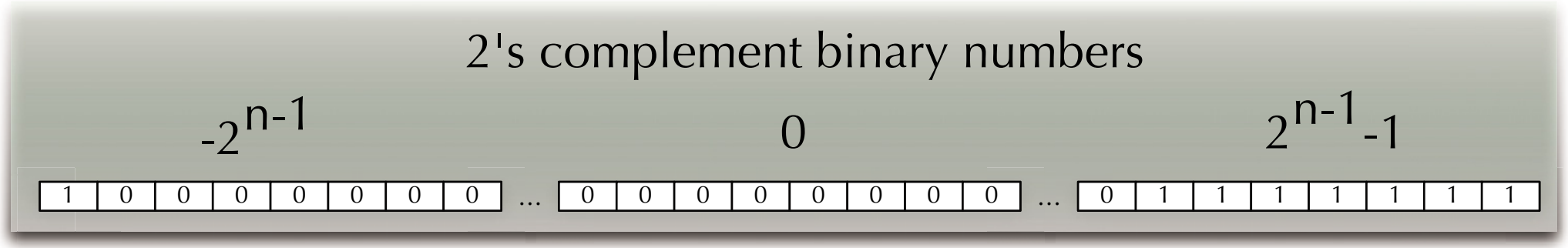



## *2's complements*

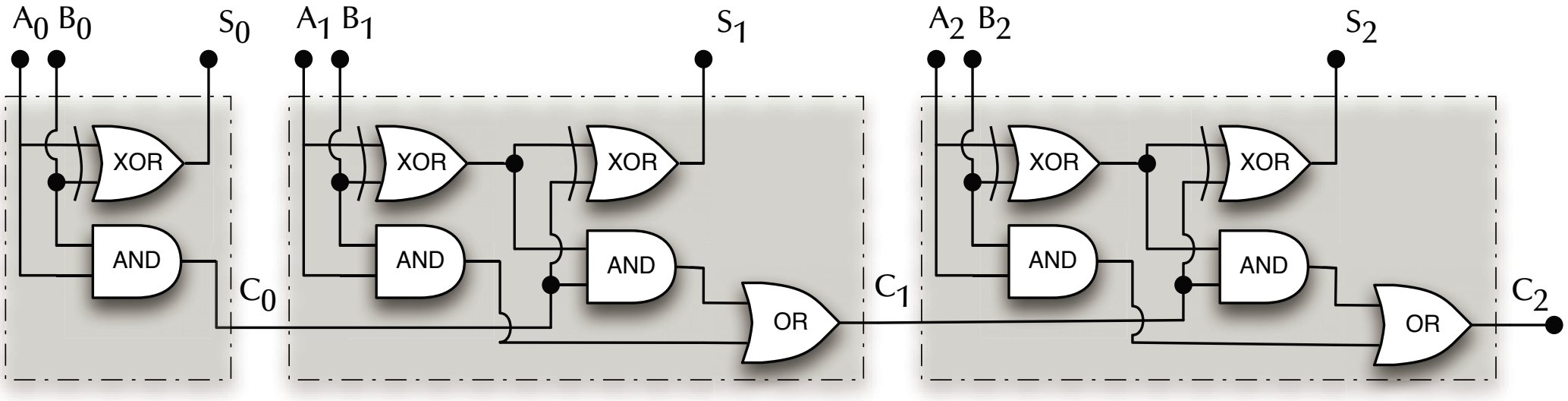
The 2's complement encoding interprets
the natural binary range $2^{n-1} \ldots 2^n - 1$
as negative numbers $-2^{n-1} \ldots -1$

*It's all in your mind!*



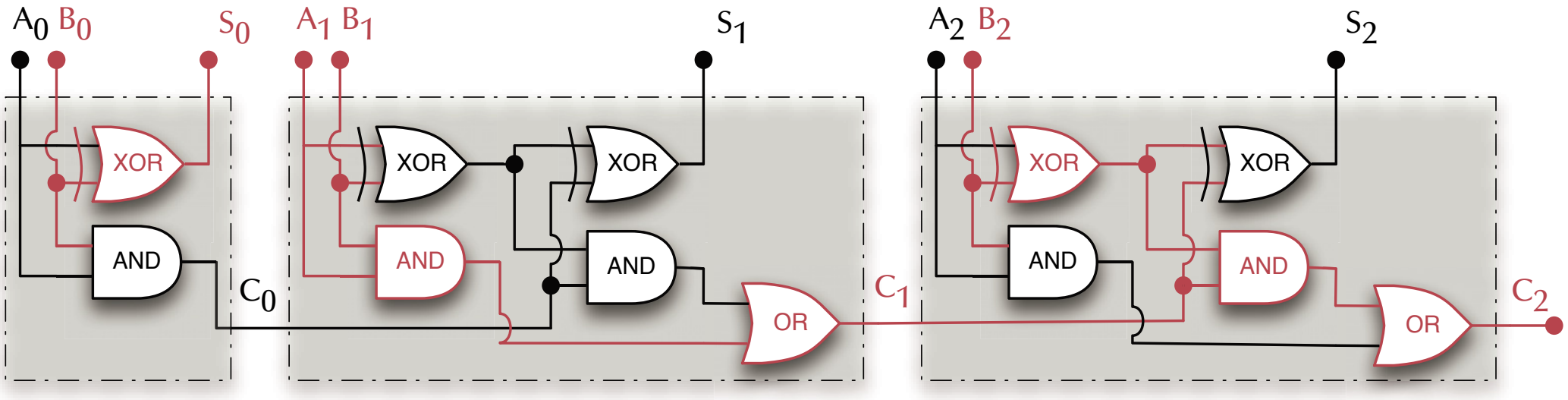



## Ripple Carry Adder




$2 - 1 = 1$ ?

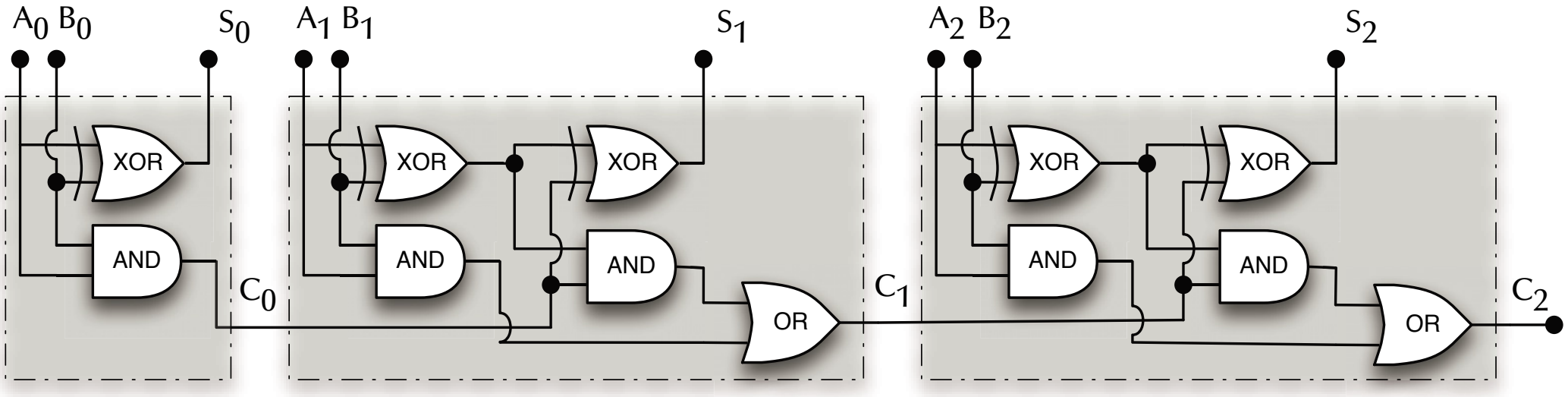
## Ripple Carry Adder




*2 - 1 = 1 !*

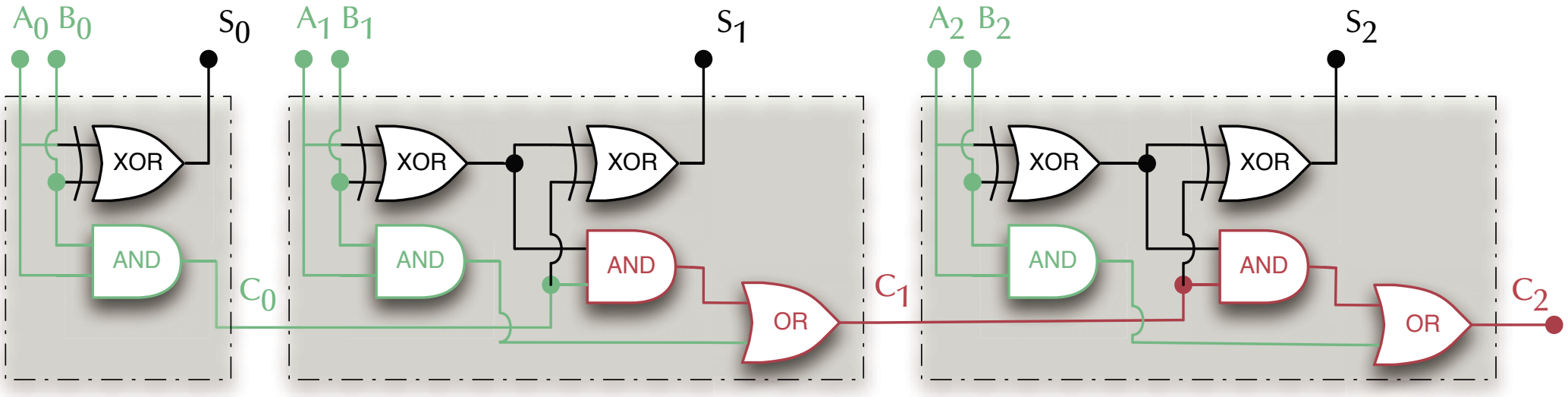
… with an overall carry-flag indicated.

## Ripple Carry Adder




*How long does it take until the last carry flag stabilizes ?*

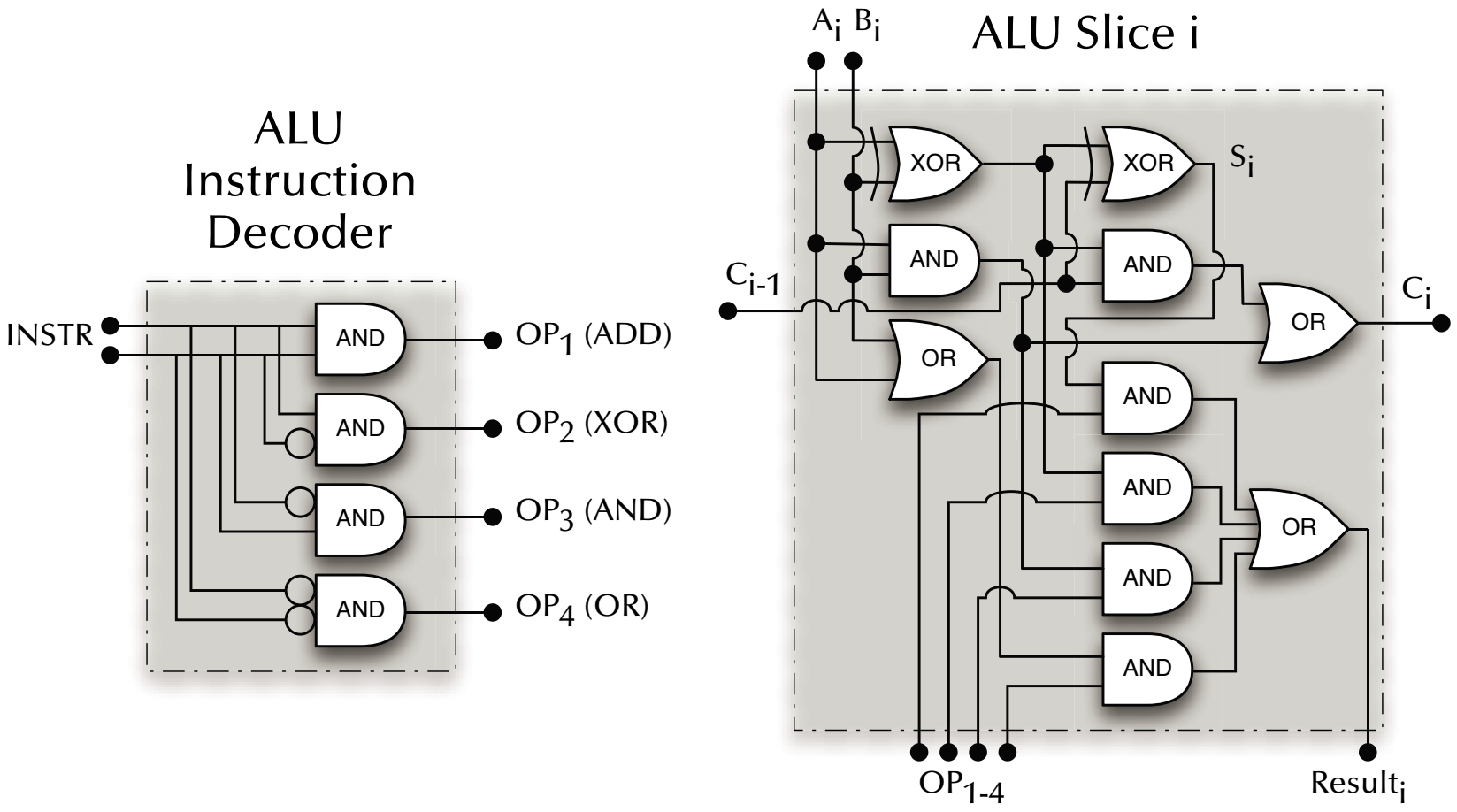
## Ripple Carry Adder




**What distinguishes the red from the green gates ?**

☞ Carry-lookahead circuitry

## *Arithmetic Logic Unit (ALU)*




A simple ALU which can ADD, XOR, AND, OR two arguments.

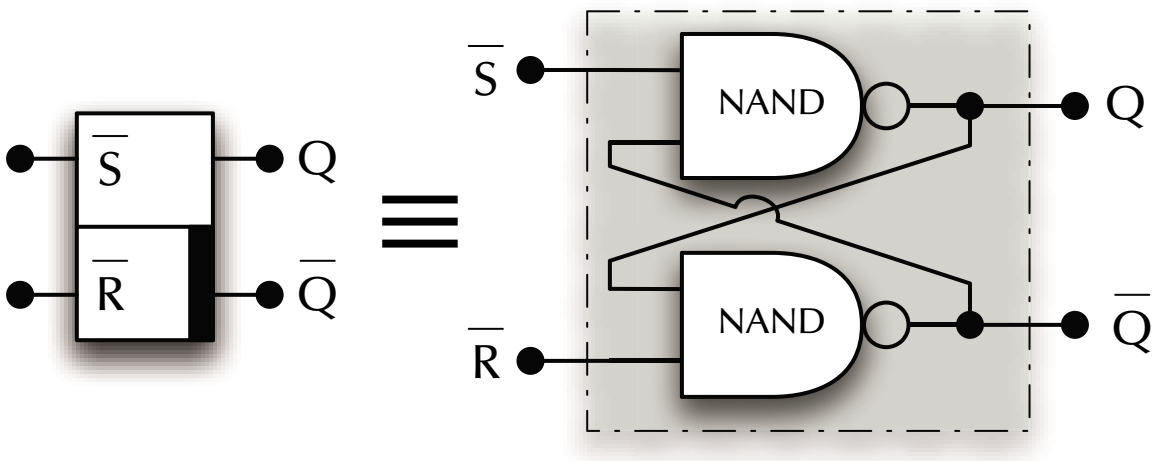
## *Towards States*

(everything up to here was combinational logic)

How do we make operations depends on:

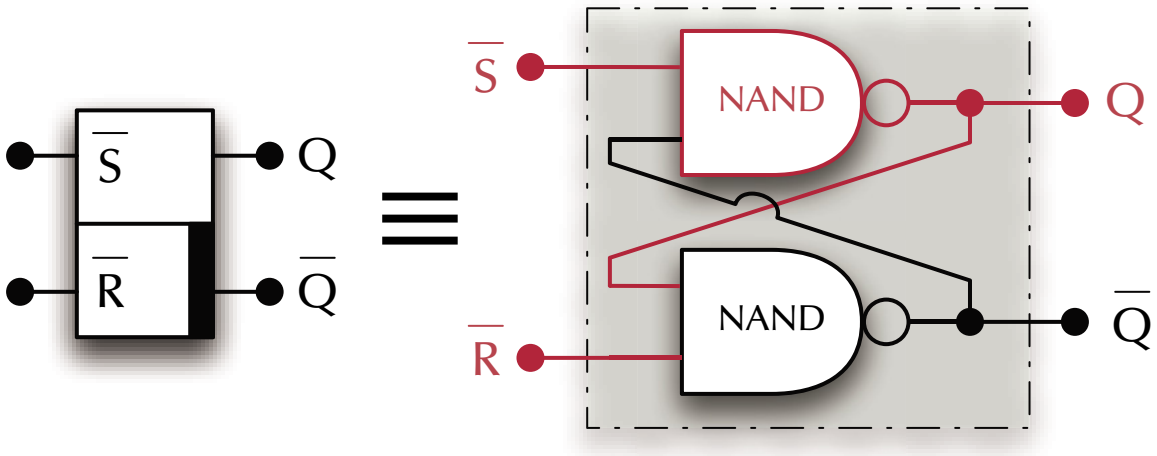
- ☞ … an overflow in the previous operation?
- ☞ … the state of the CPU?
- ☞ … a counter having reached zero?
- ☞ … two arguments having been equal?
- ☞ … etc. pp.
  - ☞ We need to hold on to some states!

## States



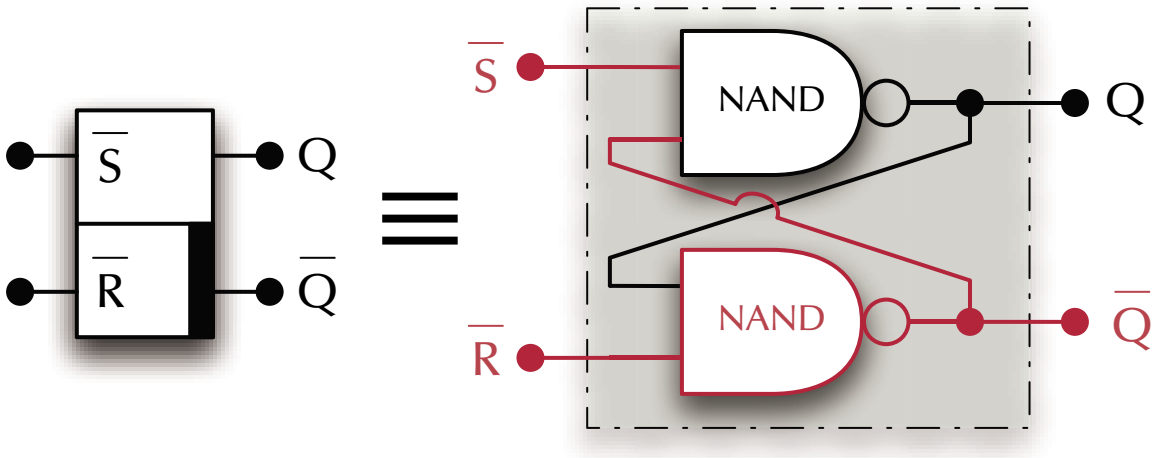

| $\overline{S}$ | $\overline{R}$ | $Q$ | $\overline{Q}$ |
|---|---|---|---|
| 0 | 0 | ? | ? |
| 0 | 1 | ? | ? |
| 1 | 0 | ? | ? |
| 1 | 1 | ? | ? |

## States



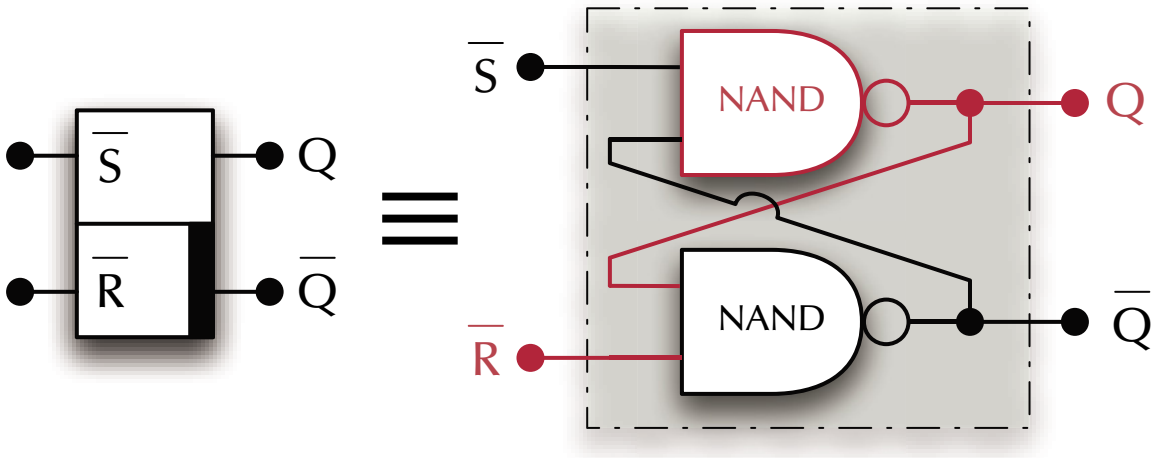

| $\overline{S}$ | $\overline{R}$ | $Q$ | $\overline{Q}$ |
|---|---|---|---|
| 0 | 0 | ? | ? |
| 0 | 1 | ? | ? |
| 1 | 0 | ? | ? |
| 1 | 1 | $Q$ | $\overline{Q}$ |

## States



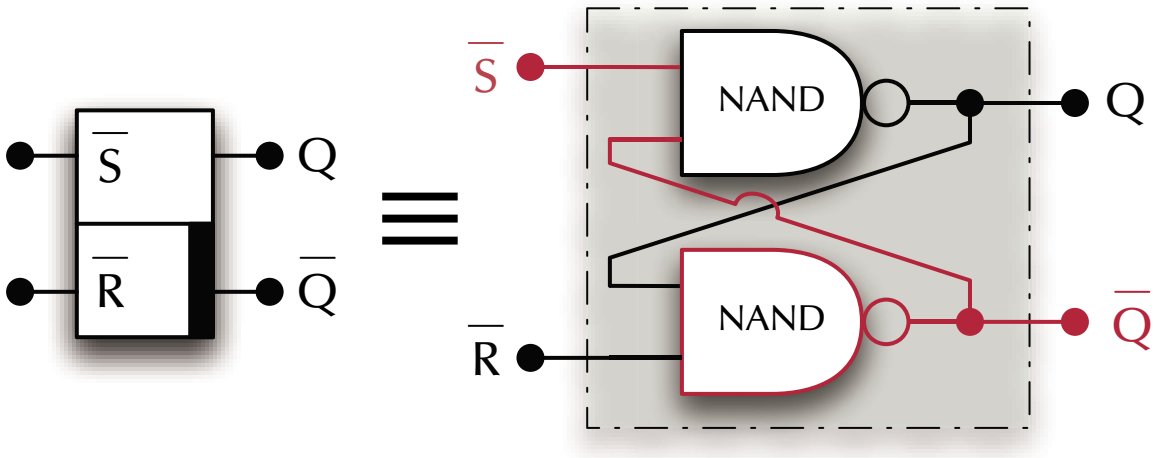

| $\overline{S}$ | $\overline{R}$ | $Q$ | $\overline{Q}$ |
|---|---|---|---|
| 0 | 0 | ? | ? |
| 0 | 1 | ? | ? |
| 1 | 0 | ? | ? |
| 1 | 1 | $Q$ | $\overline{Q}$ |

## States



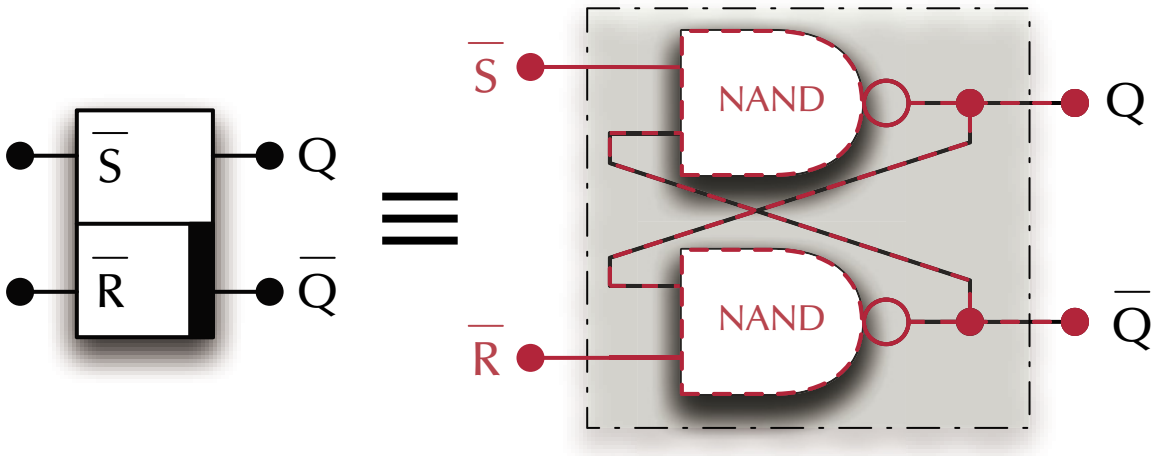

| $\overline{S}$ | $\overline{R}$ | $Q$ | $\overline{Q}$ |
|---|---|---|---|
| 0 | 0 | ? | ? |
| 0 | 1 | 1 | 0 |
| 1 | 0 | ? | ? |
| 1 | 1 | $Q$ | $\overline{Q}$ |

## States




| $\overline{S}$ | $\overline{R}$ | $Q$ | $\overline{Q}$ |
|---|---|---|---|
| 0 | 0 | ? | ? |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 1 | $Q$ | $\overline{Q}$ |

## States

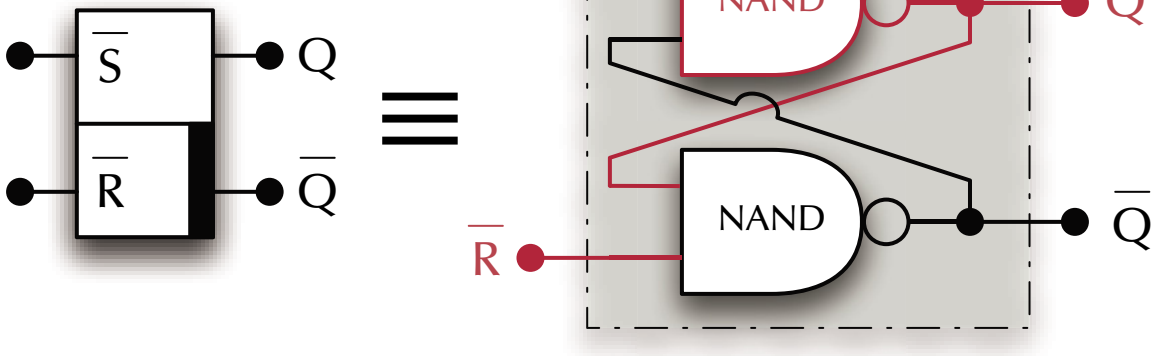



| $\overline{S}$ | $\overline{R}$ | $Q$ | $\overline{Q}$ |
|---|---|---|---|
| 0 | 0 | ½ | ½ |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 1 | $Q$ | $\overline{Q}$ |

Assuming $Q$ as well as $\overline{Q}$ to be active simultaneously may lead to instability.

## States

"S-R Flip-Flop"




| $\overline{S}$ | $\overline{R}$ | $Q$ | $\overline{Q}$ |
|---|---|---|---|
| ~~0~~ | ~~0~~ | Forbidden | |
| 0 | 1 | 1 | 0 |
| 1 | 0 | 0 | 1 |
| 1 | 1 | $Q$ | $\overline{Q}$ |

# Deriving SR Flip Flops

| $\overline{S}$ | $\overline{R}$ | $Q$ | $Q$ | | |
|---|---|---|---|---|---|
| 0 | 0 | 0 | * | | |
| 0 | 0 | 1 | * | | |
| 0 | 1 | 0 | 1 | | |
| 0 | 1 | 1 | 1 | | |
| 1 | 0 | 0 | 0 | | |
| 1 | 0 | 1 | 0 | | |
| 1 | 1 | 0 | 0 | | |
| 1 | 1 | 1 | 1 | | |

## Deriving SR Flip Flops

| $\overline{S}$ | $\overline{R}$ | $Q$ | $Q$ | $Q$ minterms |
|---|---|---|---|---|
| 0 | 0 | 0 | * | $S \wedge R \wedge \overline{Q}$ |
| 0 | 0 | 1 | * | $S \wedge R \wedge Q$ |
| 0 | 1 | 0 | 1 | $S \wedge \overline{R} \wedge \overline{Q}$ |
| 0 | 1 | 1 | 1 | $S \wedge \overline{R} \wedge Q$ |
| 1 | 0 | 0 | 0 | |
| 1 | 0 | 1 | 0 | |
| 1 | 1 | 0 | 0 | |
| 1 | 1 | 1 | 1 | $\overline{S} \wedge \overline{R} \wedge Q$ |



## Deriving SR Flip Flops

| $\overline{S}$ | $\overline{R}$ | $Q$ | $Q$ | $Q$ minterms | Simplified |
|---|---|---|---|---|---|
| 0 | 0 | 0 | * | $S \wedge R \wedge \overline{Q}$ | $S$ |
| 0 | 0 | 1 | * | $S \wedge R \wedge Q$ | |
| 0 | 1 | 0 | 1 | $S \wedge \overline{R} \wedge \overline{Q}$ | |
| 0 | 1 | 1 | 1 | $S \wedge \overline{R} \wedge Q$ | $\overline{R} \wedge Q$ |
| 1 | 0 | 0 | 0 | | |
| 1 | 0 | 1 | 0 | | |
| 1 | 1 | 0 | 0 | | |
| 1 | 1 | 1 | 1 | $\overline{S} \wedge \overline{R} \wedge Q$ | |

☞ $Q = S \vee (\overline{R} \wedge Q)$



## Deriving SR Flip Flops

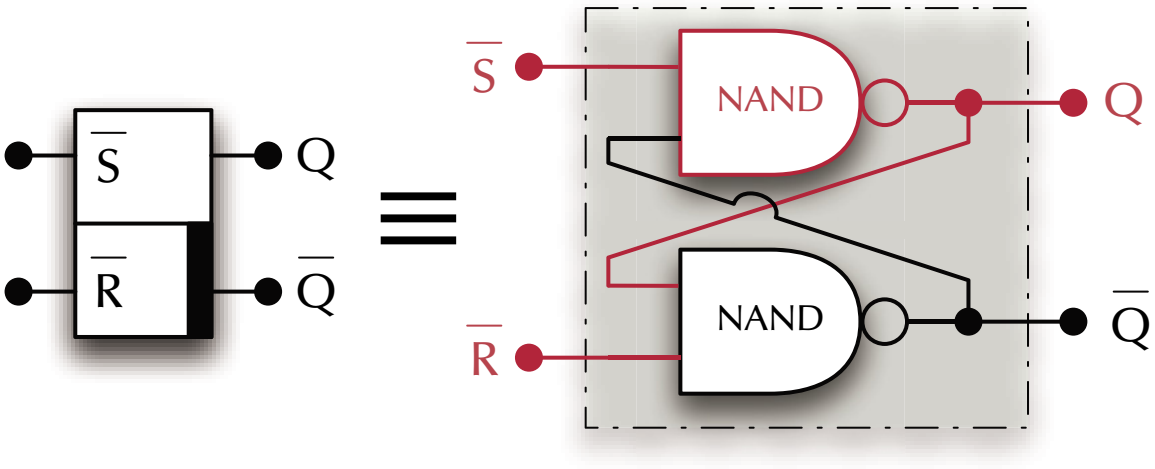
| $\overline{S}$ | $\overline{R}$ | $Q$ | $Q$ | $Q$ minterms | Simplified |
|---|---|---|---|---|---|
| 0 | 0 | 0 | * | $S \wedge R \wedge \overline{Q}$ | $S$ |
| 0 | 0 | 1 | * | $S \wedge R \wedge Q$ | |
| 0 | 1 | 0 | 1 | $S \wedge \overline{R} \wedge \overline{Q}$ | |
| 0 | 1 | 1 | 1 | $S \wedge \overline{R} \wedge Q$ | $\overline{R} \wedge Q$ |
| 1 | 0 | 0 | 0 | | |
| 1 | 0 | 1 | 0 | | |
| 1 | 1 | 0 | 0 | | |
| 1 | 1 | 1 | 1 | $\overline{S} \wedge \overline{R} \wedge Q$ | |

☞ $Q = S \vee (\overline{R} \wedge Q) = \overline{\overline{S} \wedge \overline{\overline{R} \wedge Q}}$

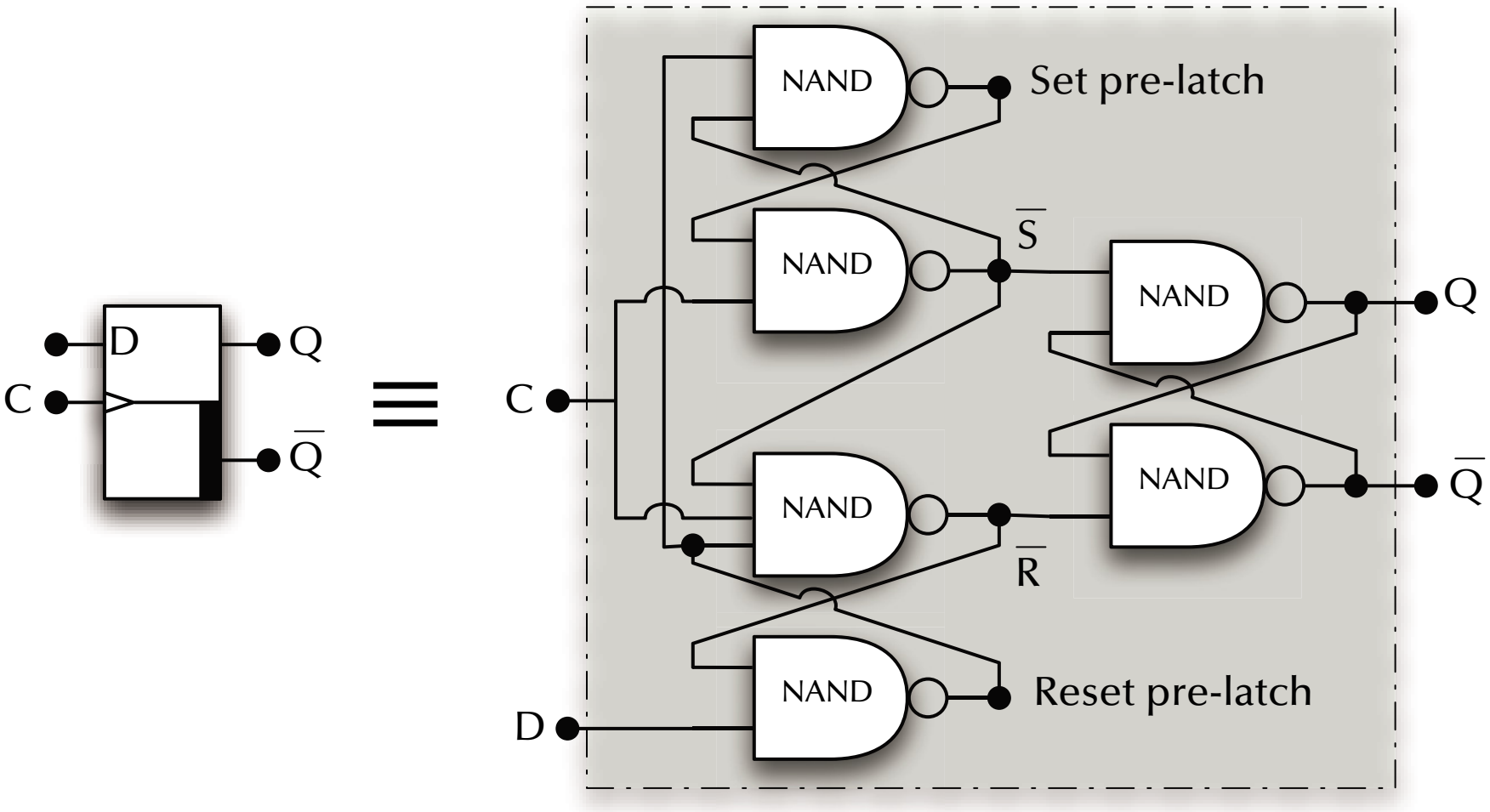
## Deriving SR Flip Flops

| $\overline{S}$ | $\overline{R}$ | $Q$ | $Q$ | $Q$ minterms | Simplified |
|---|---|---|---|---|---|
| 0 | 0 | 0 | * | $S \wedge R \wedge \overline{Q}$ | |
| 0 | 0 | 1 | * | $S \wedge R \wedge Q$ | $S$ |
| 0 | 1 | 0 | 1 | $S \wedge \overline{R} \wedge \overline{Q}$ | |
| 0 | 1 | 1 | 1 | $S \wedge \overline{R} \wedge Q$ | |
| 1 | 0 | 0 | 0 | | |
| 1 | 0 | 1 | 0 | | $\overline{R} \wedge Q$ |
| 1 | 1 | 0 | 0 | | |
| 1 | 1 | 1 | 1 | $\overline{S} \wedge \overline{R} \wedge Q$ | |

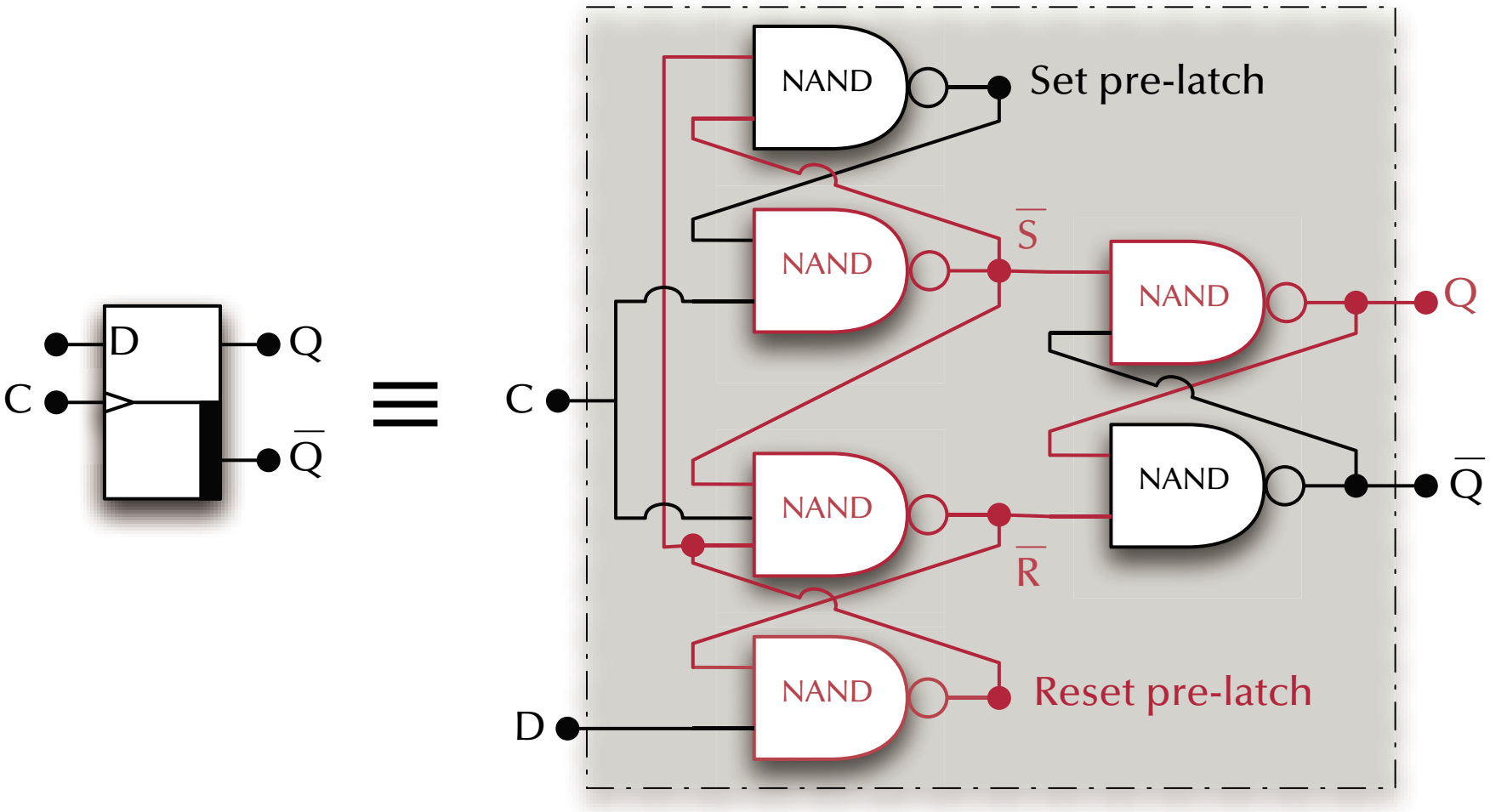
☞ $Q = S \vee (\overline{R} \wedge Q) = \overline{\overline{S} \wedge \overline{\overline{R} \wedge Q}}$



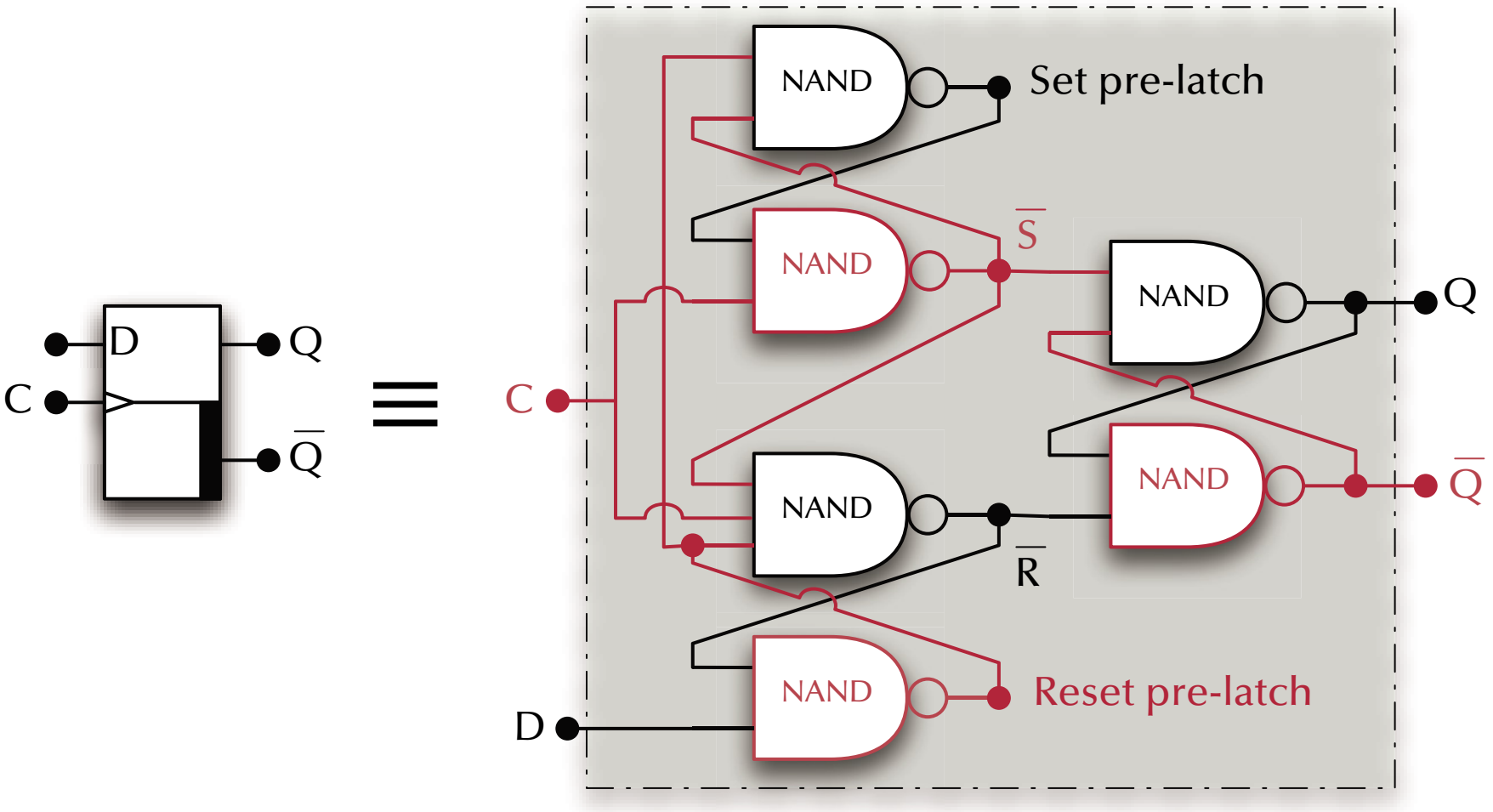
## D Flip-Flop


D
C
Q
$\overline{Q}$
≡
C
D
NAND
NAND
NAND
NAND
NAND
NAND
Set pre-latch
Reset pre-latch
$\overline{S}$
$\overline{R}$
Q
$\overline{Q}$

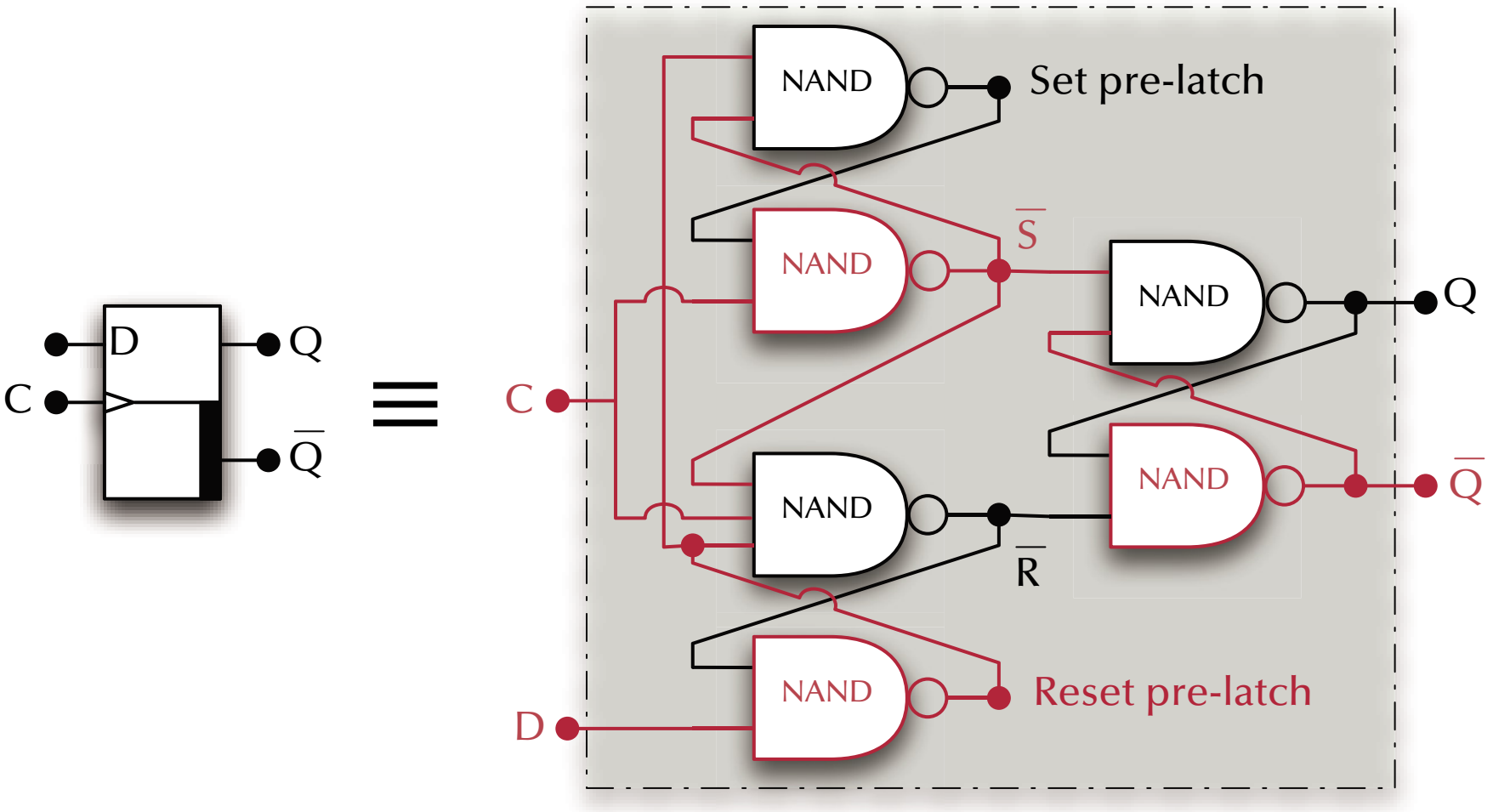
## D Flip-Flop


D
C
Q
$\overline{Q}$
≡
C
D
NAND
NAND
NAND
NAND
NAND
NAND
Set pre-latch
Reset pre-latch
$\overline{S}$
$\overline{R}$
Q
$\overline{Q}$

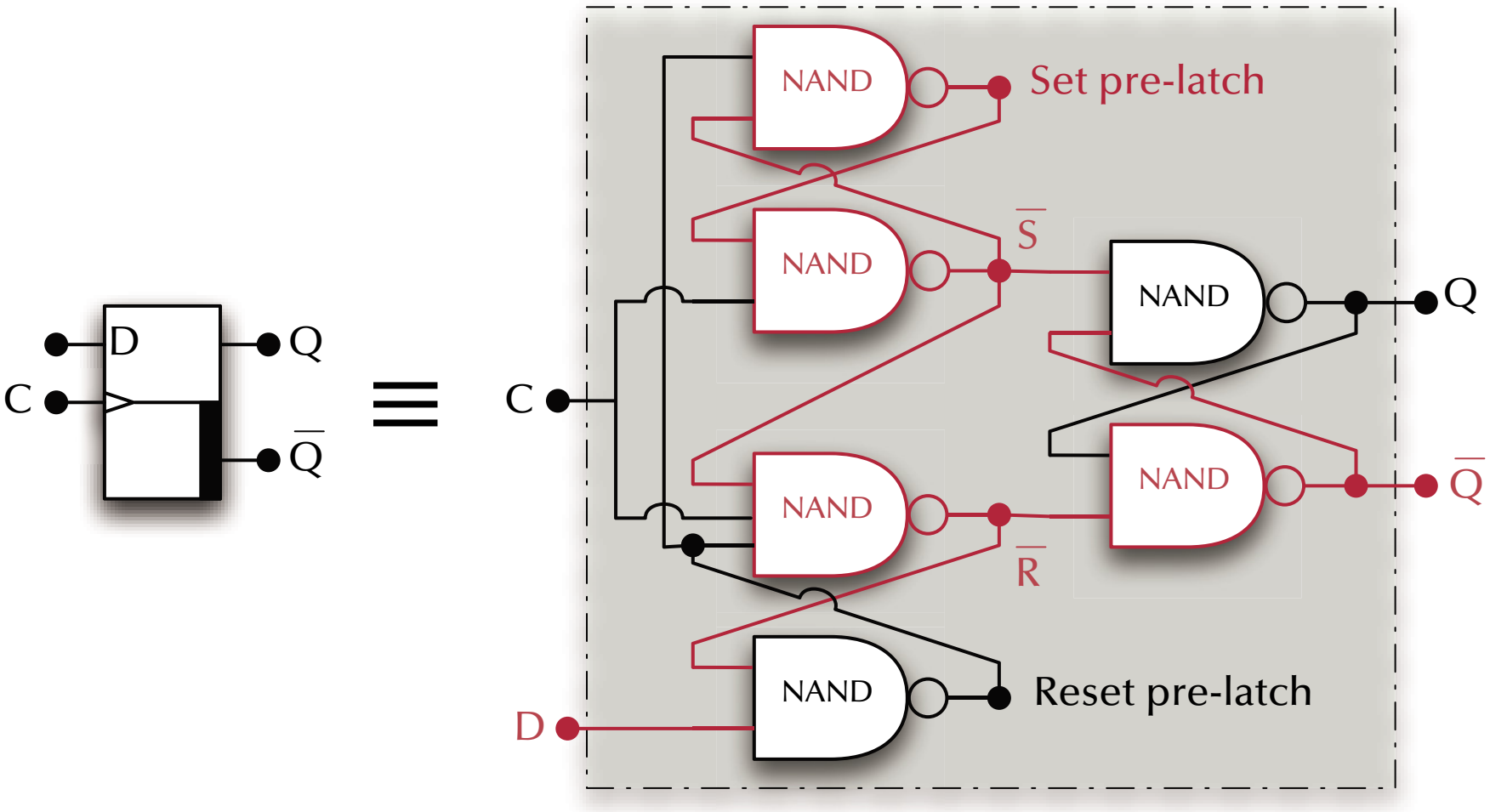
## D Flip-Flop


D
C
Q
$\overline{Q}$
≡
NAND
Set pre-latch
$\overline{S}$
NAND
NAND
Q
C
NAND
NAND
$\overline{Q}$
$\overline{R}$
NAND
Reset pre-latch
D

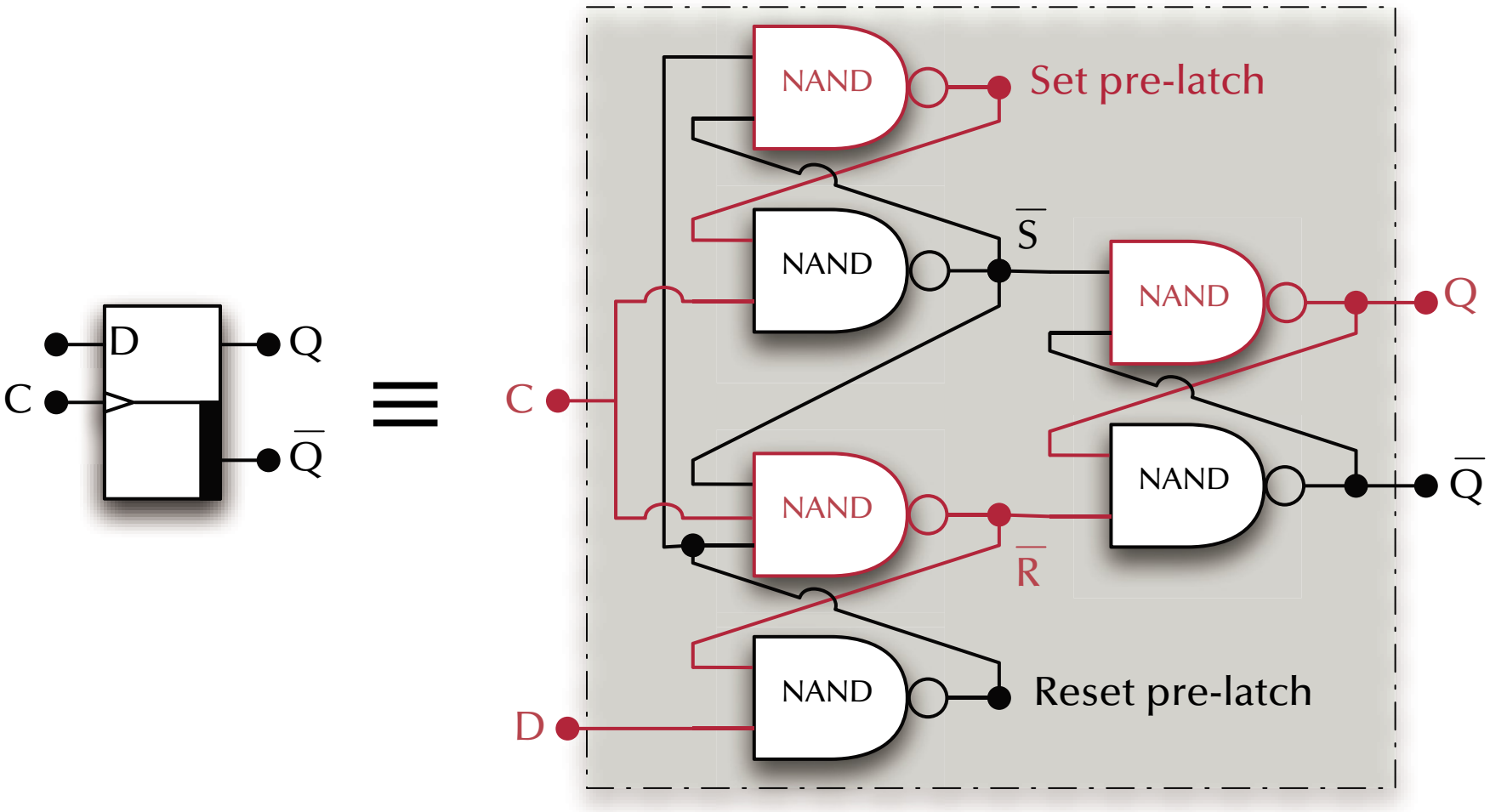
## D Flip-Flop


D
C
Q
$\overline{Q}$
≡
NAND
Set pre-latch
$\overline{S}$
NAND
NAND
Q
C
NAND
NAND
$\overline{Q}$
$\overline{R}$
NAND
Reset pre-latch
D

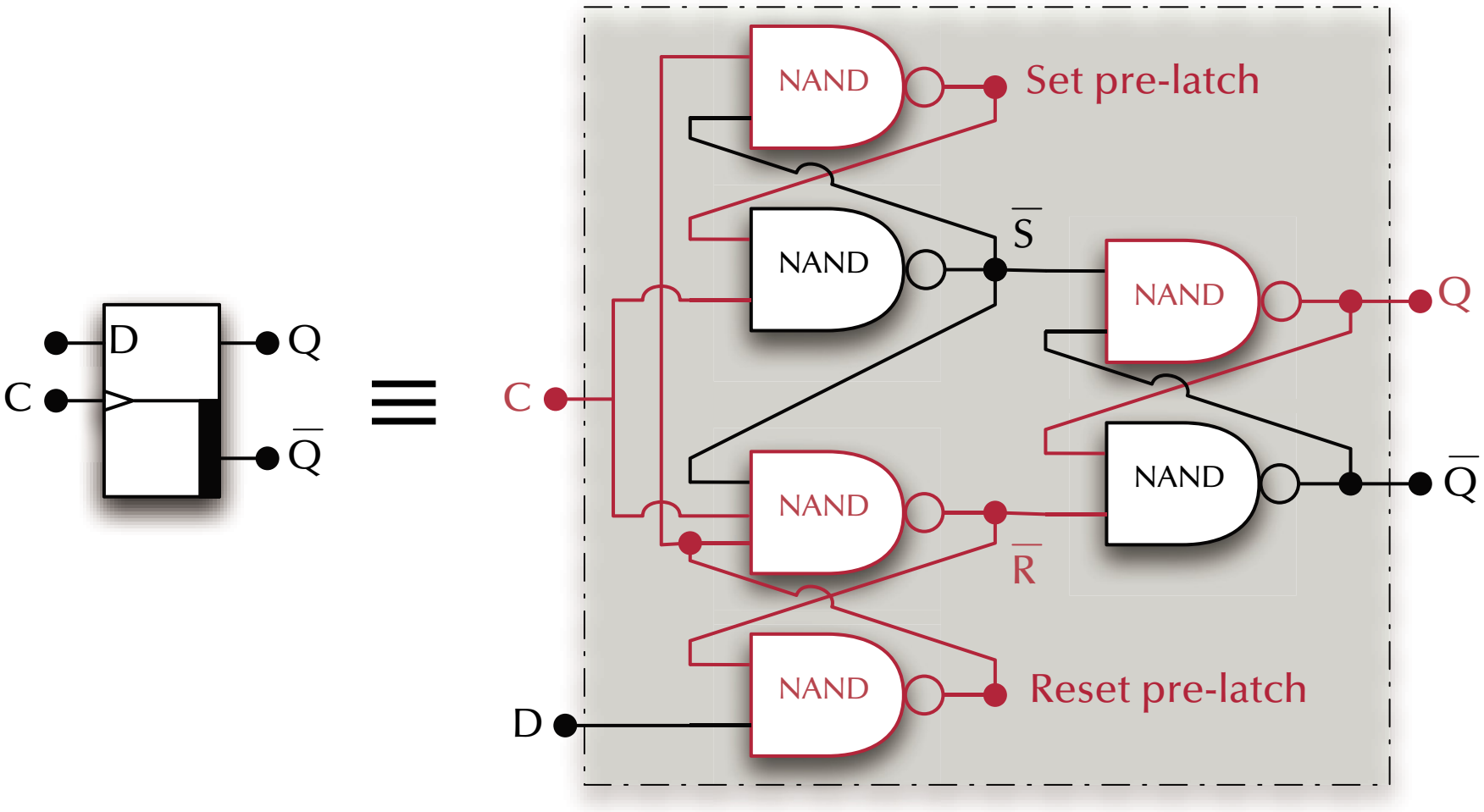
## D Flip-Flop


D
C
Q
$\overline{Q}$
≡
C
D
NAND
Set pre-latch
NAND
$\overline{S}$
NAND
Q
NAND
$\overline{Q}$
NAND
$\overline{R}$
NAND
Reset pre-latch

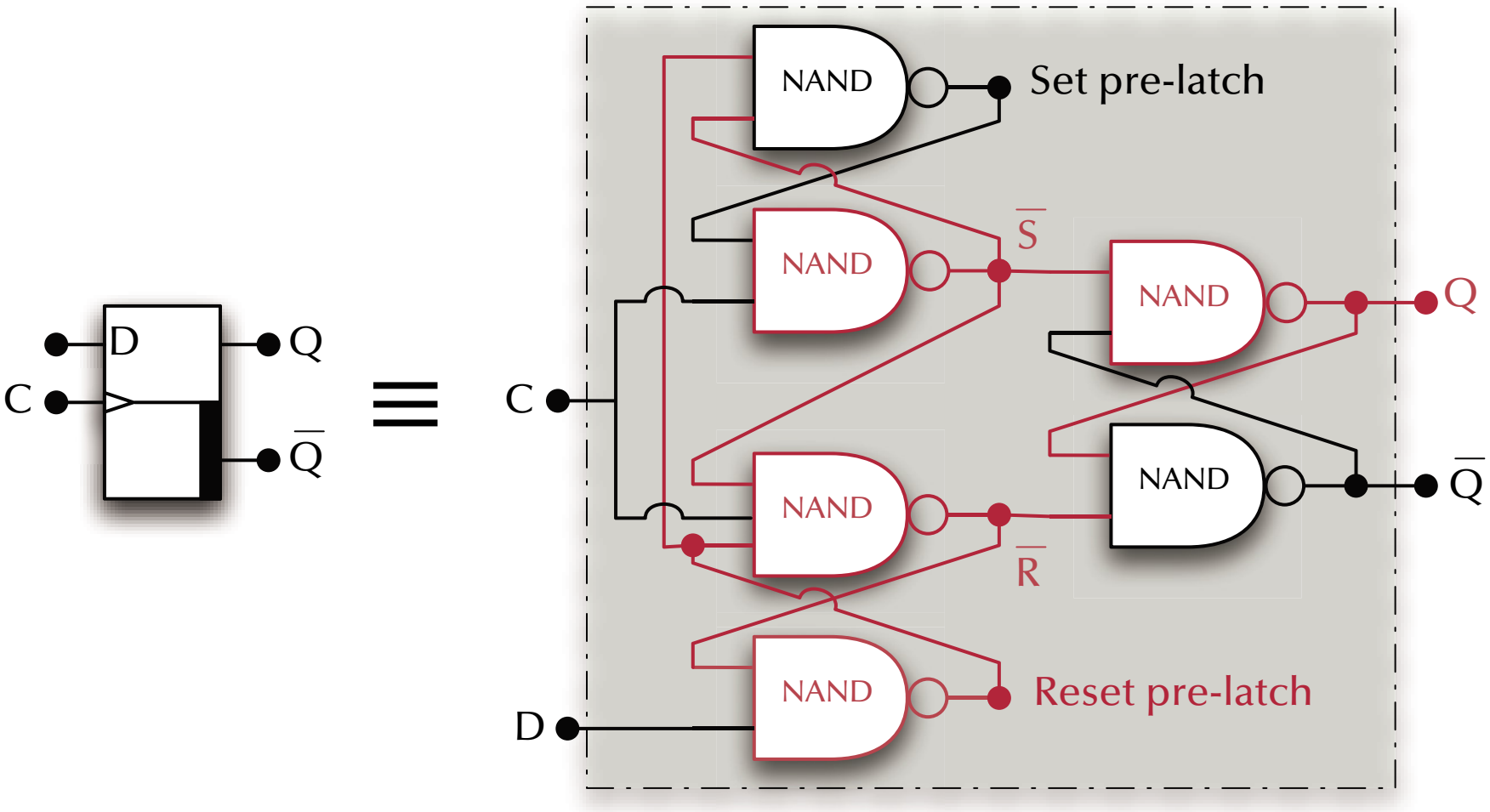
## D Flip-Flop


D
C
Q
$\overline{Q}$
≡
NAND
Set pre-latch
$\overline{S}$
NAND
NAND
Q
C
NAND
NAND
$\overline{Q}$
$\overline{R}$
NAND
Reset pre-latch
D

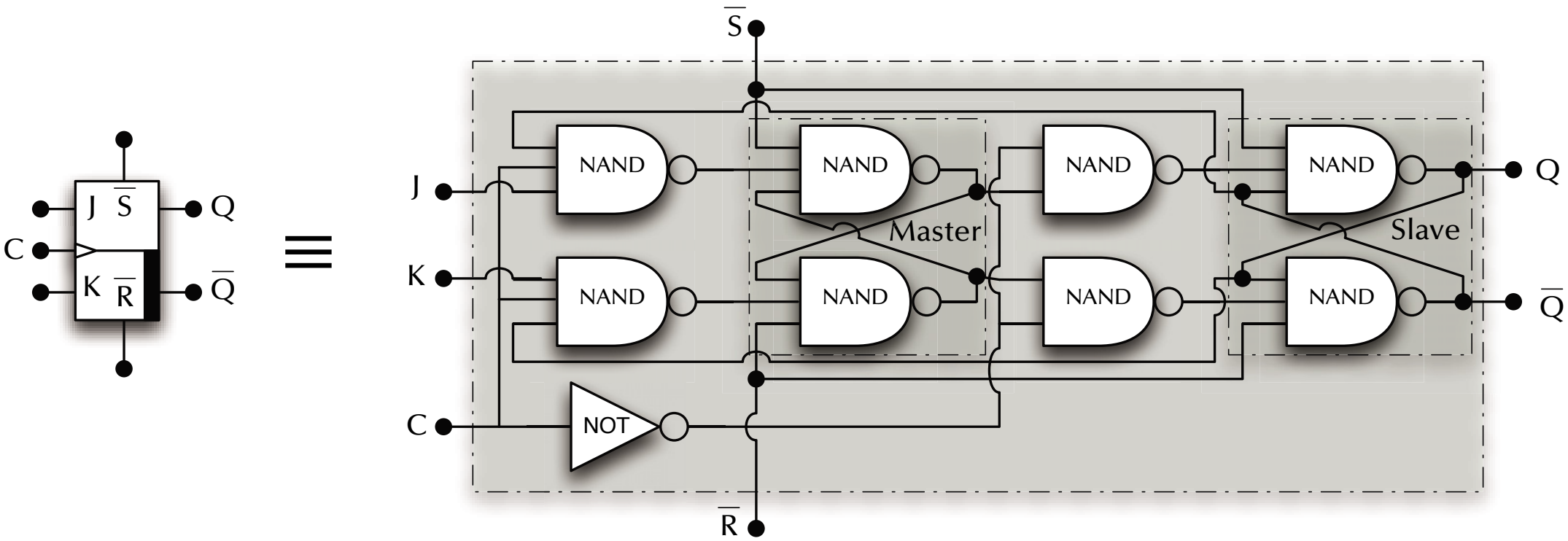
## D Flip-Flop


D
C
Q
$\overline{Q}$
≡
NAND
Set pre-latch
$\overline{S}$
NAND
NAND
Q
C
NAND
$\overline{Q}$
NAND
$\overline{R}$
NAND
Reset pre-latch
D

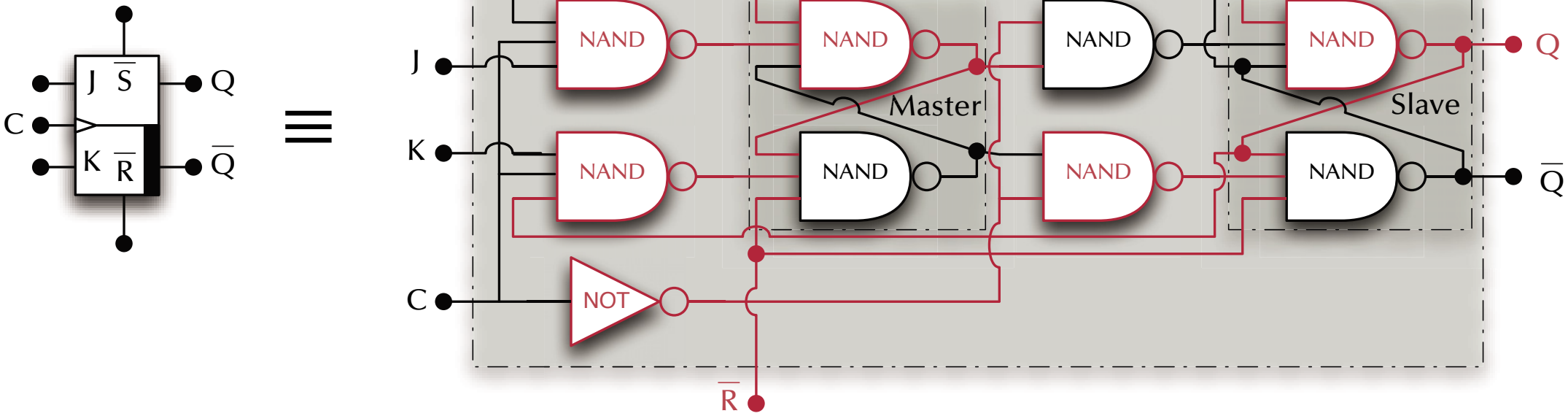
## D Flip-Flop


D
C
Q
$\overline{Q}$
≡
C
D
NAND
NAND
NAND
NAND
NAND
NAND
Set pre-latch
Reset pre-latch
$\overline{S}$
$\overline{R}$
Q
$\overline{Q}$

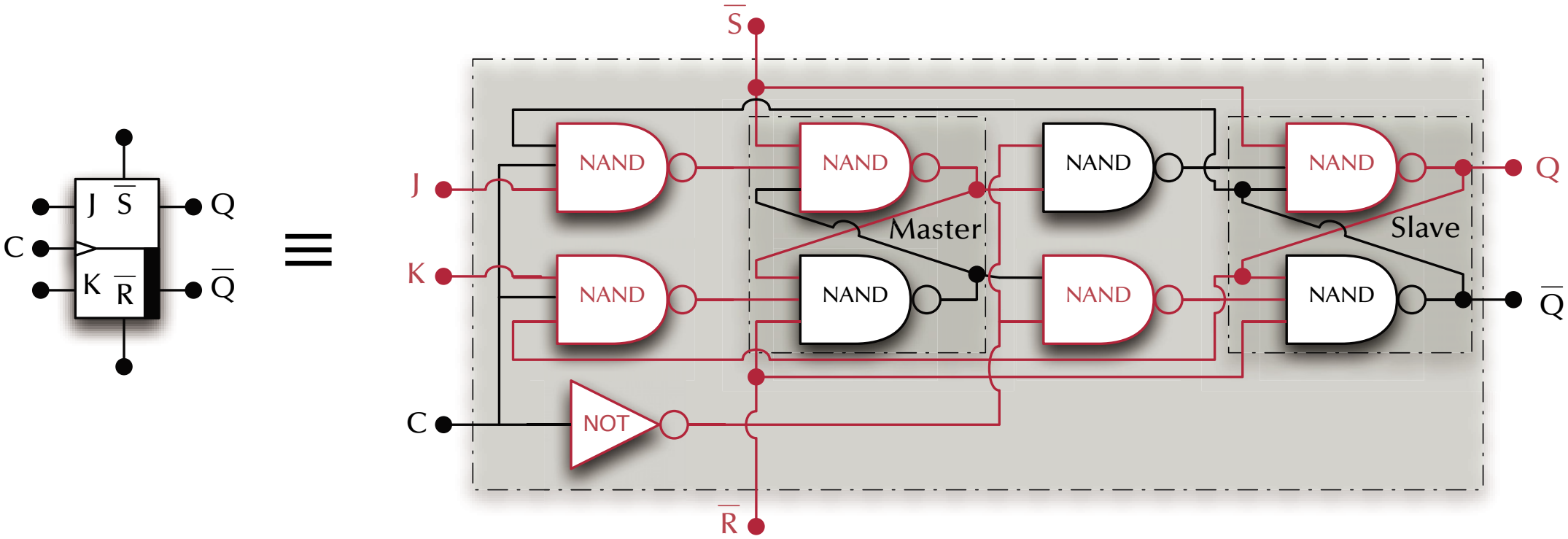
## Master-Slave JK Flip-Flop



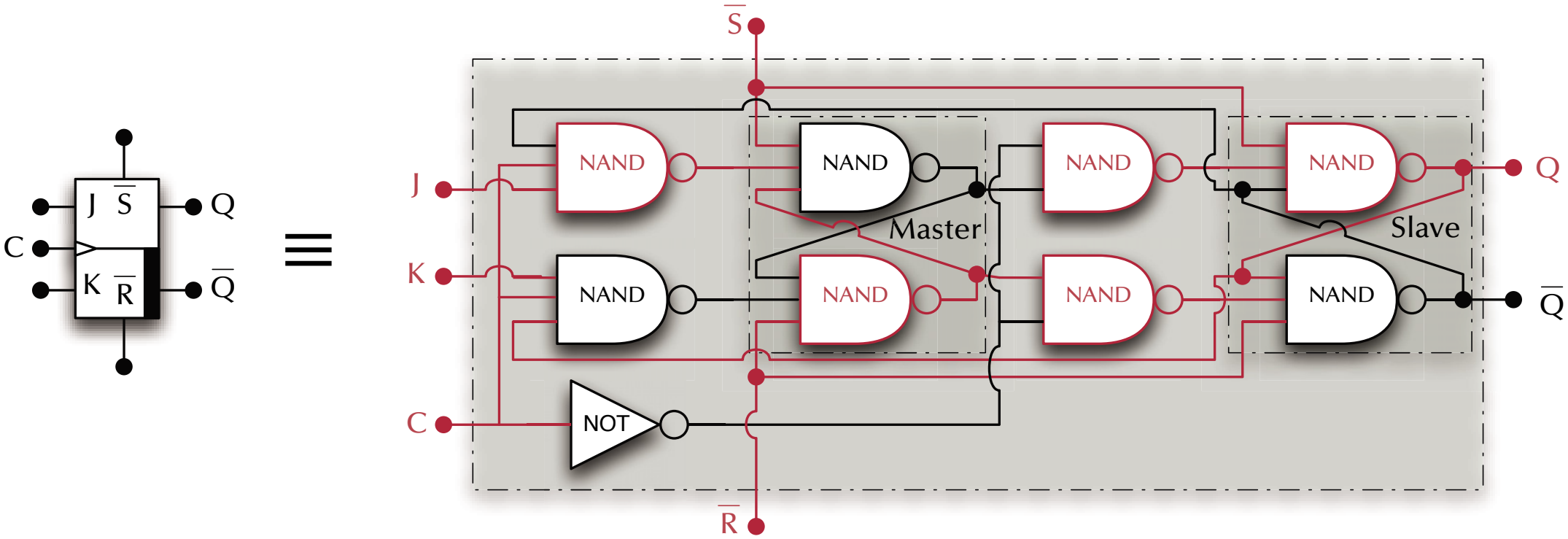
## Master-Slave JK Flip-Flop



## Master-Slave JK Flip-Flop

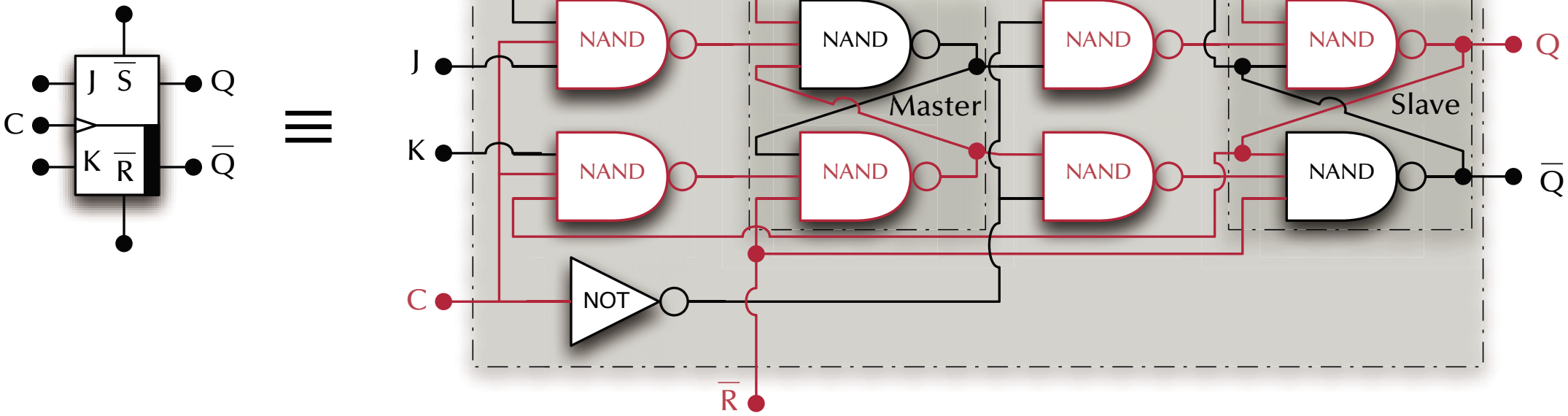


## Master-Slave JK Flip-Flop

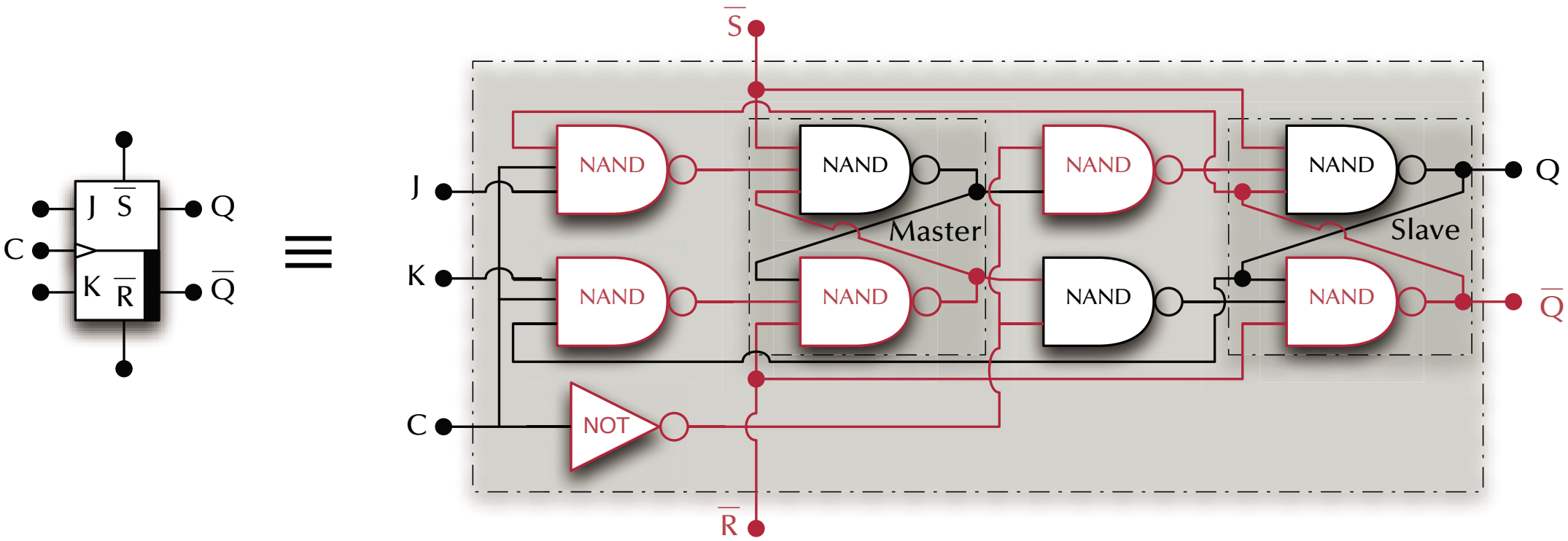



Master is reset on the rising clock edge

## Master-Slave JK Flip-Flop

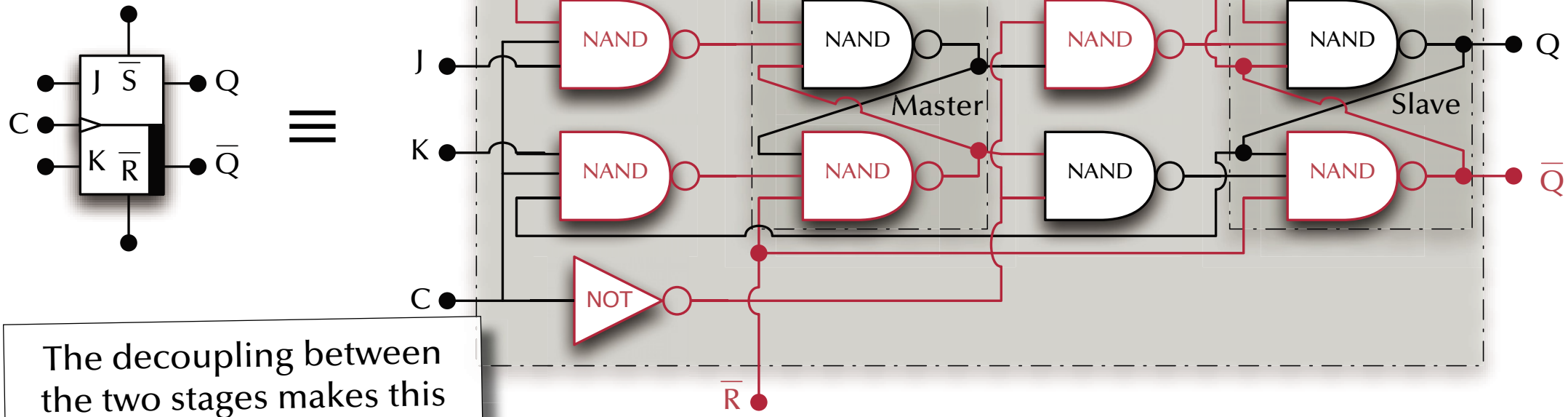


## Master-Slave JK Flip-Flop




Slave follows on the falling clock edge

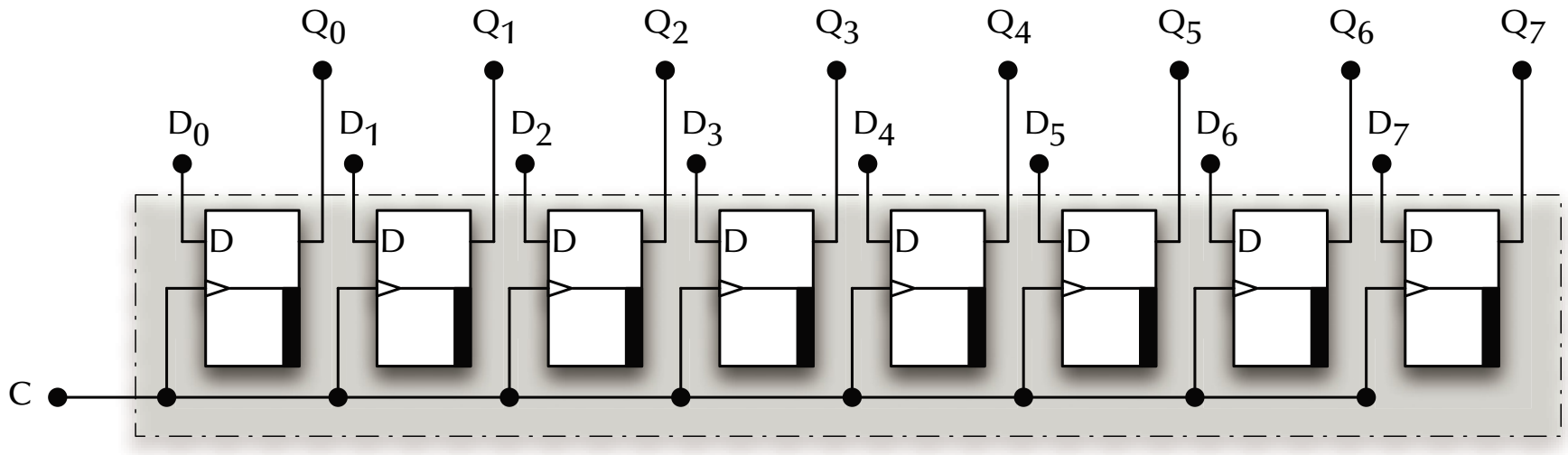
## Master-Slave JK Flip-Flop




The decoupling between the two stages makes this flip-flop race free – even in JK-toggle mode.

Slave follows on the falling clock edge

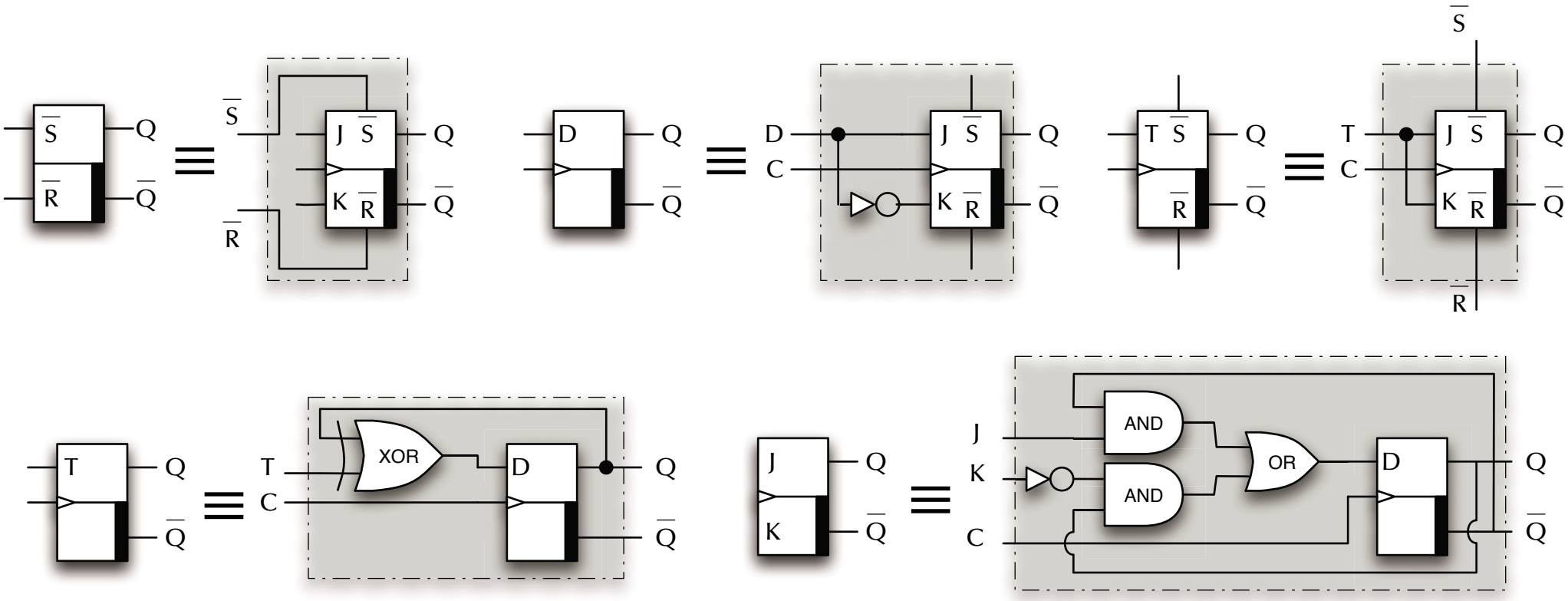
## *Register*




Could serve as a generic, fast storage inside the CPU (general register)

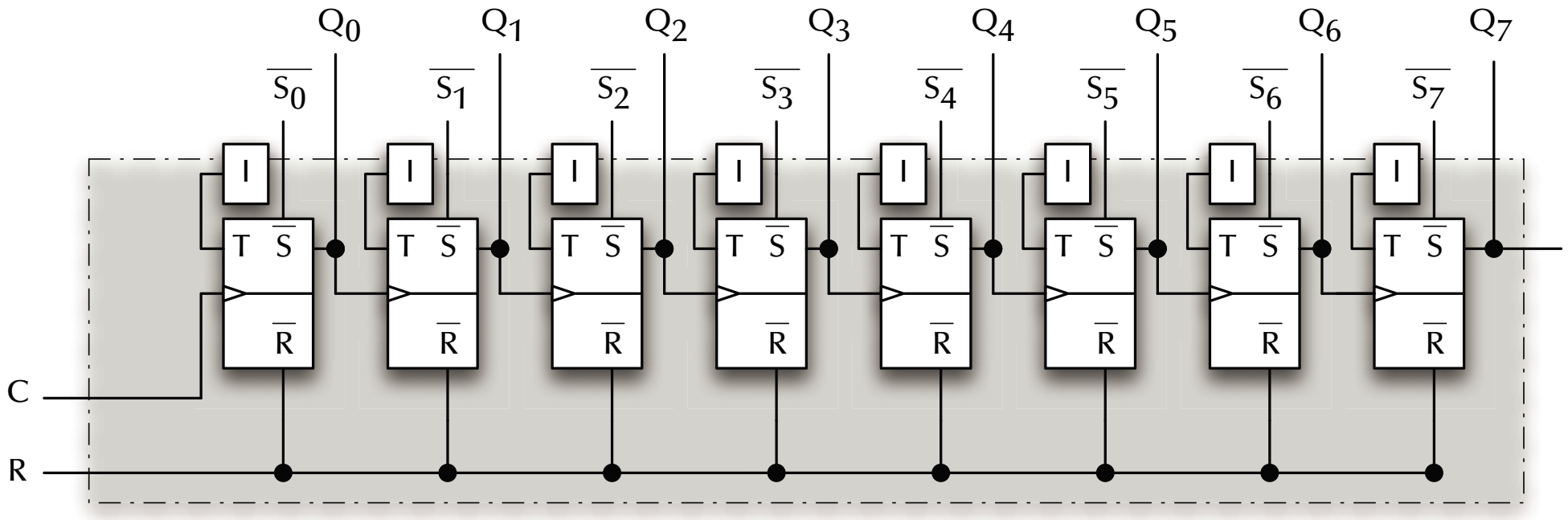
Or to hold internal states (e.g. ALU overflow) of the CPU which are used by e.g. branching instructions.

## Toggle Flip-Flop



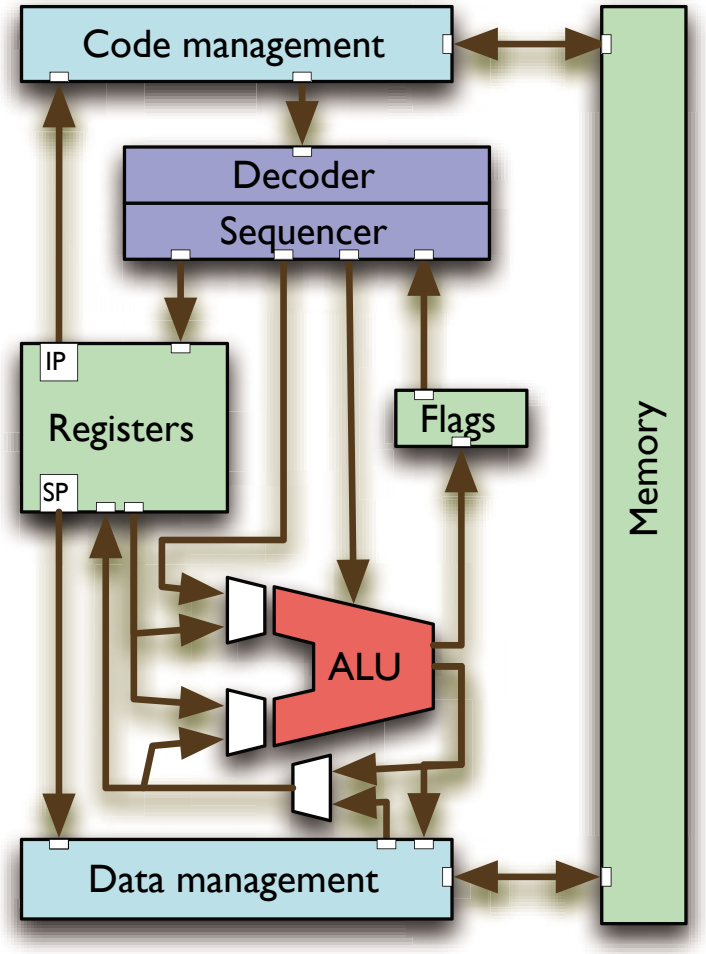

Toggle Flip-Flops change state with every clock cycle.

## Counter




Your controller has many counters which can e.g. be used to delay operations without the need to execute instructions.

## *Simple CPU Architecture*



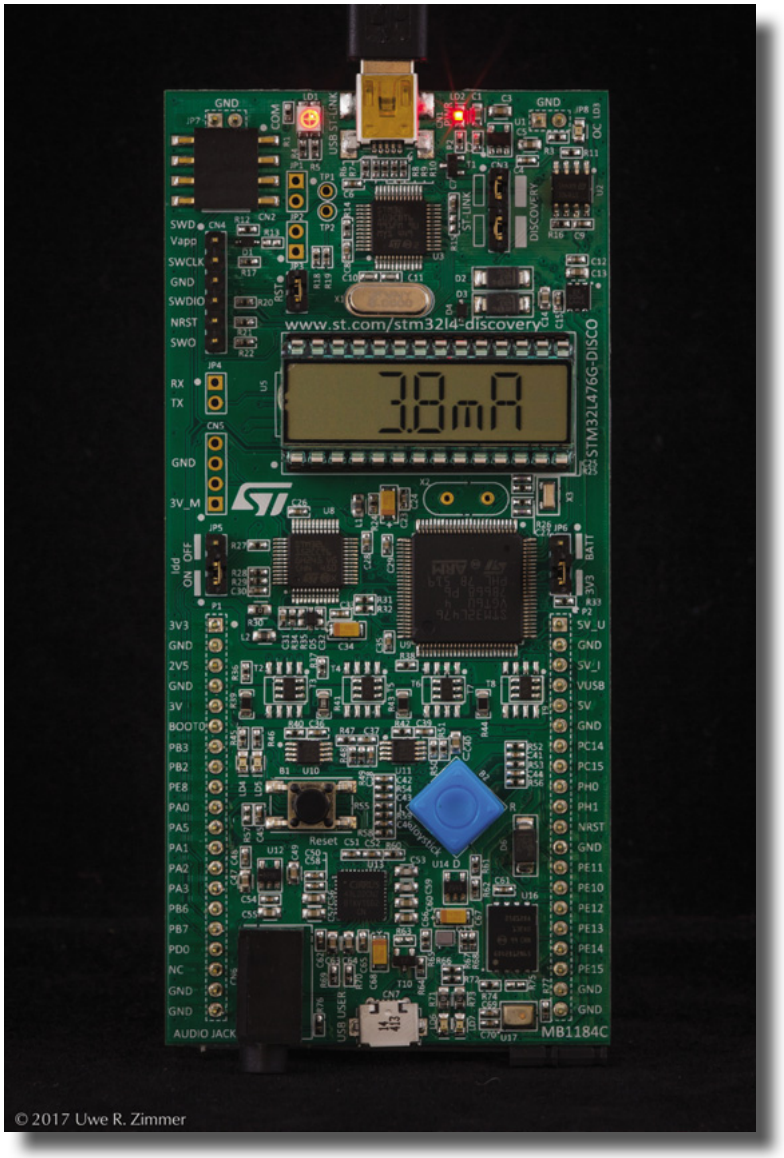

You can already build most of the components of a CPU by now.

(The most essential missing component is the **sequencer** which is a specialized state-machine.)

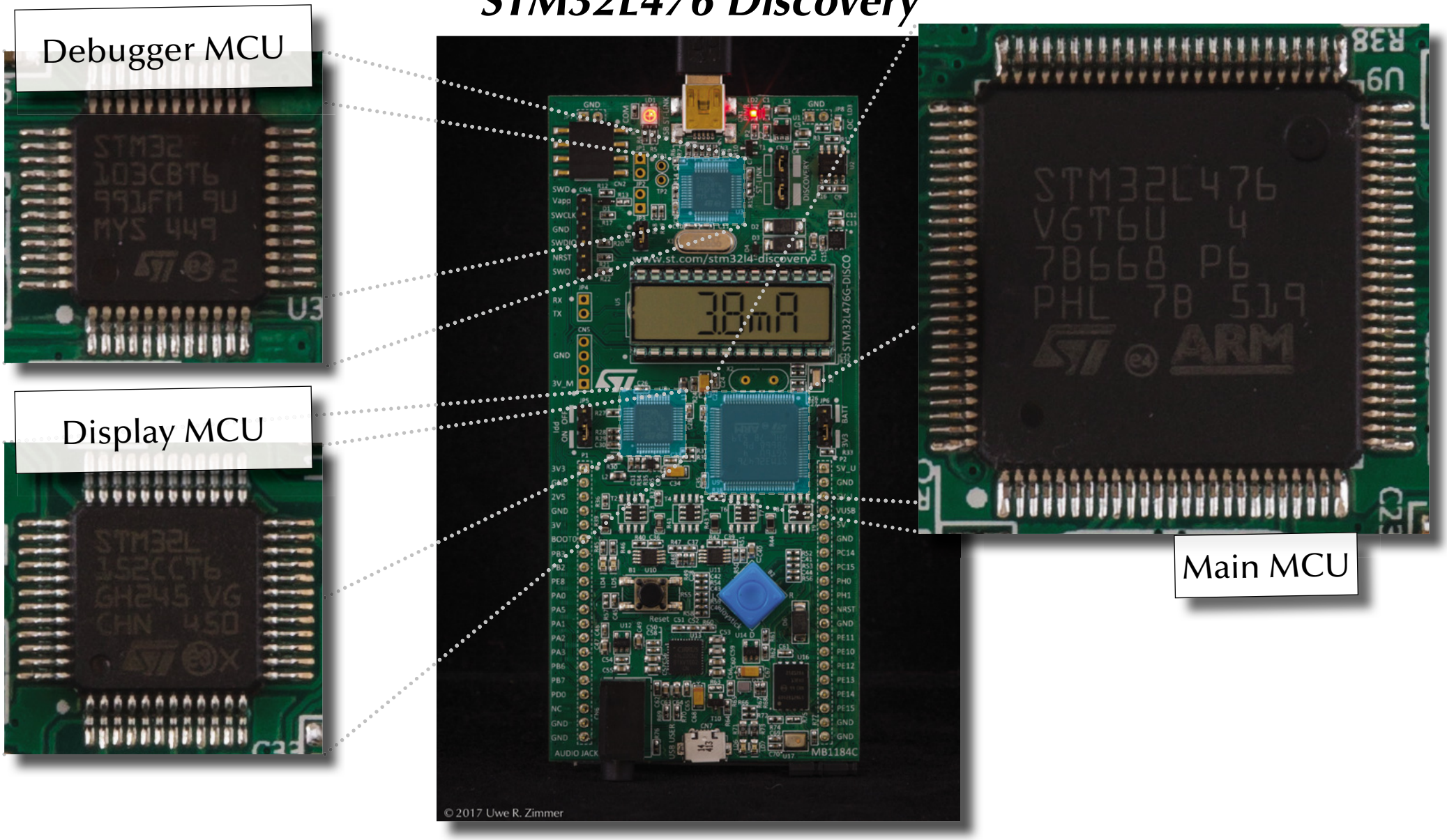
We will come back to the CPU architectures towards the end of the course.

☞ The next chapter will be about programming a CPU at machine level.

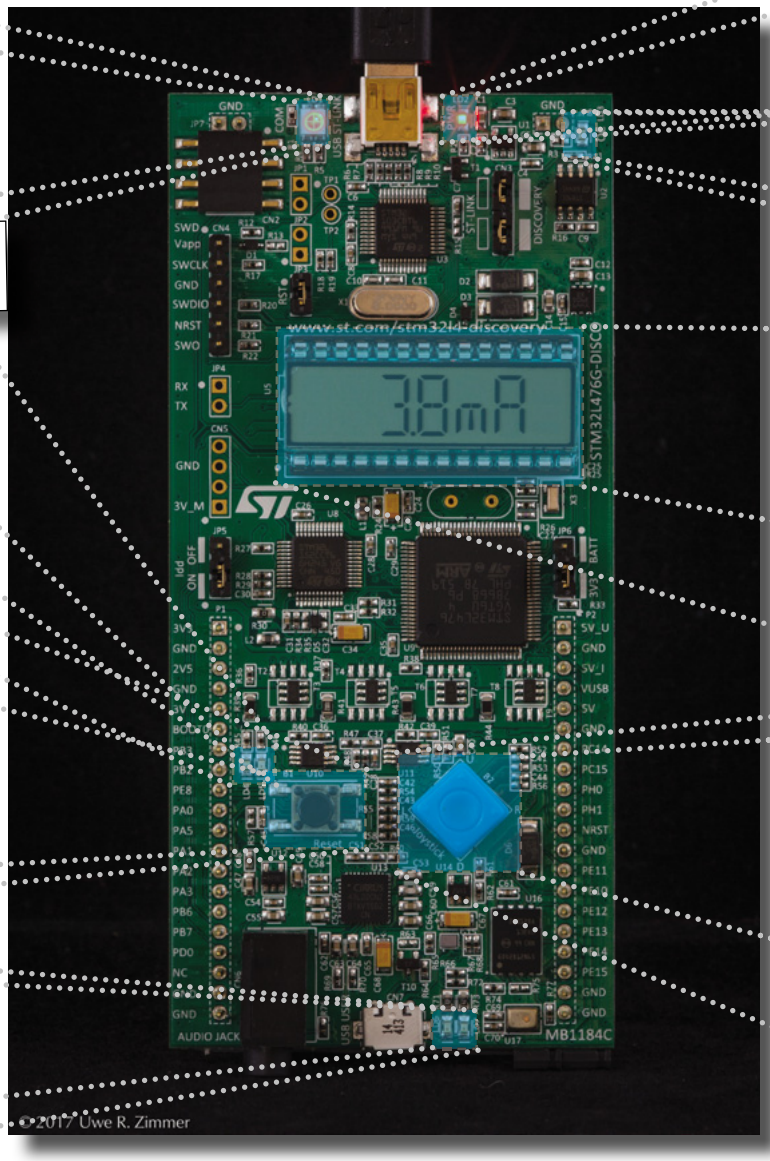
## STM32L476 Discovery



## *STM32L476 Discovery*

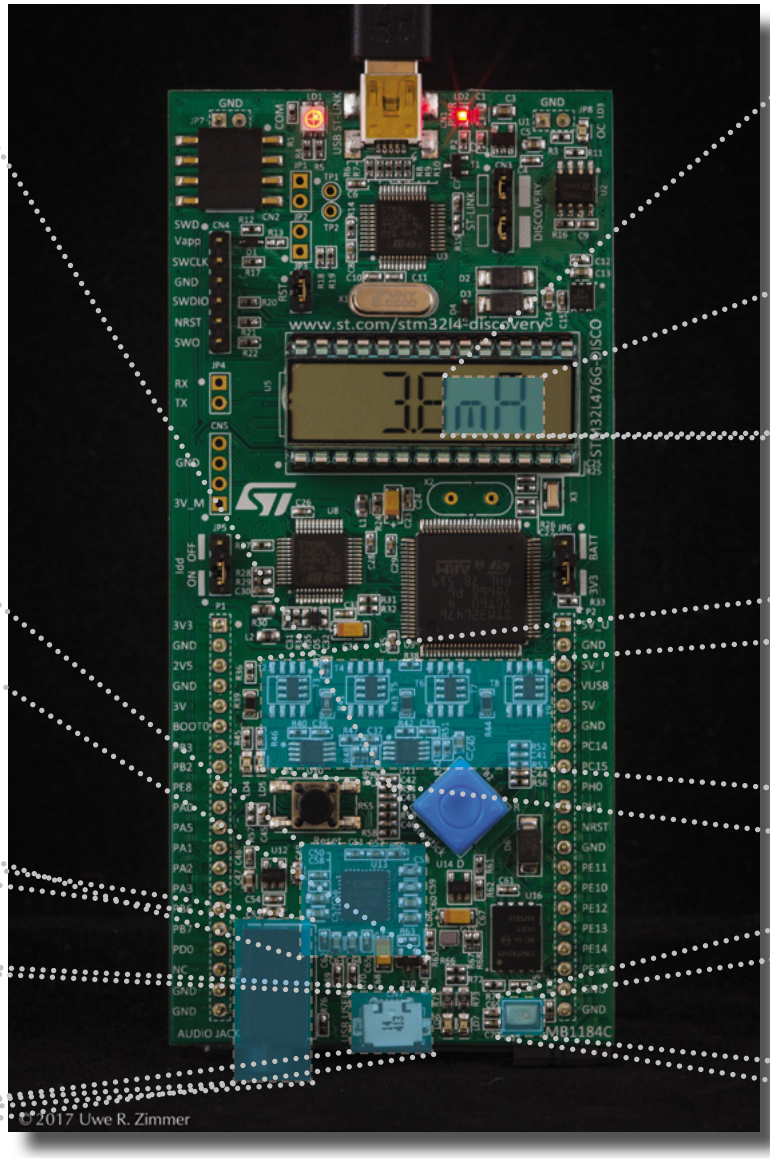


## STM32L476 Discovery



## *STM32L476 Discovery*

Multiplexed 24 bit
ΣΔ-DAConverter with
stereo power amp

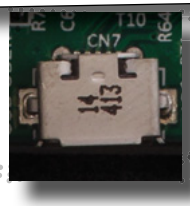

"9 axis" motion sensor
(underneath display):

- 3 axis accelerometer
- 3 axis gyroscope
- 3 axis magnetometer

Current meter to MCU
60 nA … 50 mA


Headphone jack

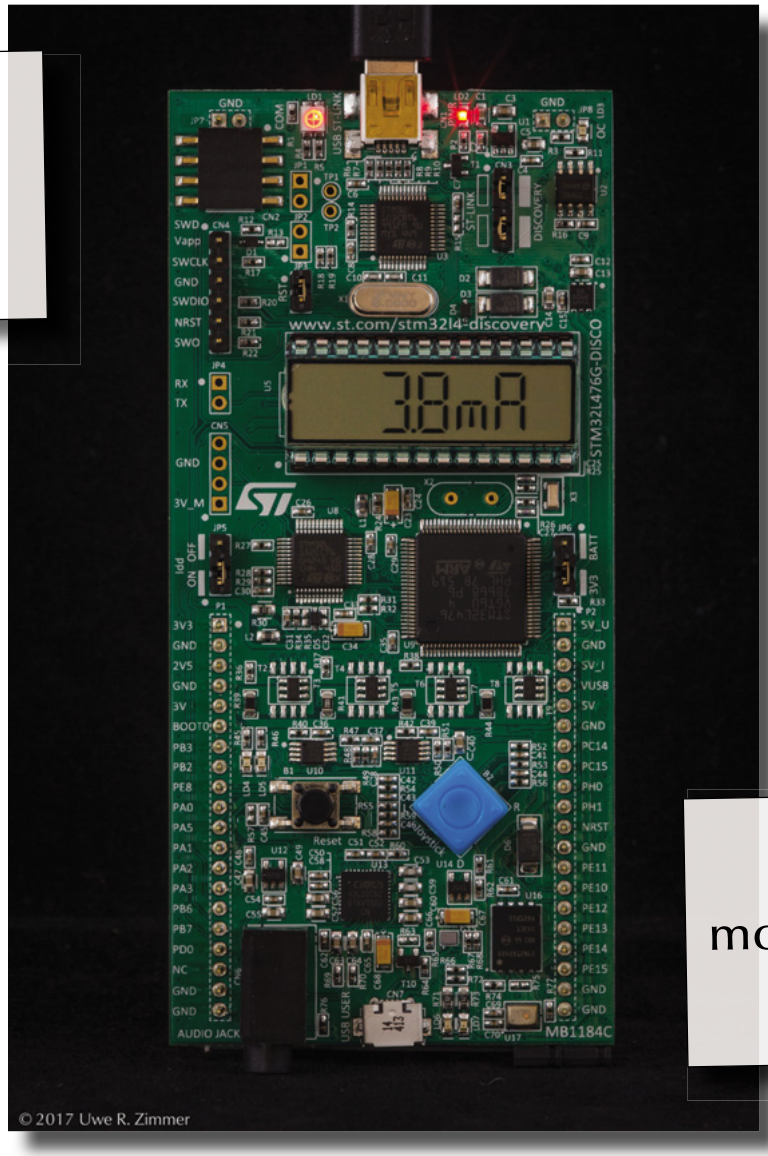
USB OTG




Microphone

## *STM32L476 Discovery*

There is a lot more hardware here than you could possibly master in one semester …




… and you will master a lot more about CPUs at the end the course than you think now.

## *Summary*

# *Digital Logic*

- **Boolean Algebra**
  - Truth tables and Boolean operations
  - Minterms and simplifying expressions
- **Combinational Logic**
  - Logic gates
  - Numbers
  - Adders, ALU
- **State-oriented Logic**
  - Flip-Flops, registers and counters
- **CPU Architecture**